BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

DEVELOPMENT OF AN LLM RED-TEAMING TOOLKIT

VÝVOJ TOOLKITU PRO RED-TEAMING VELKÝCH JAZYKOVÝCH MODELŮ (LLM)

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **ADAM VESELÝ**
AUTOR PRÁCE

SUPERVISOR **Ing. JAKUB REŠ**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172326

Institut:	Department of Intelligent Systems (DITS)
Student:	**Veselý Adam**
Programme:	Information Technology
Title:	**Development of an LLM Red-Teaming Toolkit: Design, Implementation, and Evaluation**
Category:	Security
Academic year:	2025/26

Assignment:

1. Review the current state of research in the field of red-teaming large language models (LLMs), including attack tactics (jailbreaking, prompt injection, multi-turn adversarial prompts), threat types, and current approaches to evaluating the security of language models.
2. Research existing open-source tools and frameworks for LLM red-teaming (e.g., PyRIT, garak, MAD-MAX), their architectures, advantages, and limitations.
3. Design the architecture of a new modular red-teaming toolkit for LLMs, enabling the detection of LLM weaknesses, the definition of different types of attacks, the automation of test scenarios, and the evaluation of results.
4. Implement a prototype of the proposed system that will be able to generate and evaluate attacks on LLM (e.g., LLaMA 3, GPT-OSS), including support for both manual and automatic modes.
5. Test the functionality of the system on a selected model. Focus on the success rate of attacks and the robustness of the testing environment.

Literature:

- Purpura et al., Building Safe GenAI Applications: An Overview of Red Teaming for LLMs, https://arxiv.org/abs/2503.01742
- Schoepf et al., MAD-MAX: Modular Adversarial Red Teaming of LLMs, https://arxiv.org/abs/2503.06253
- Belaire et al., Automatic LLM Red Teaming, https://arxiv.org/abs/2508.04451v1

Requirements for the semestral defence:
Items 1 to 3.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Reš Jakub, Ing.**
Head of Department:	Kočí Radek, Ing., Ph.D.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	3.11.2025

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

VESELÝ, Adam. *Development of an LLM red-teaming toolkit.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Jakub Reš,

Development of an LLM red-teaming toolkit

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Mr. Ing. Jakub Reš. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT to correct spelling and other language mistakes.

. . . . . .
Adam Veselý
May 4, 2026

Acknowledgements

I would like to thank my supervisor Ing. Jakub Reš for his guidance and support throughout the development of this thesis. I also appreciate the assistance provided by my colleagues and friends who contributed their insights and expertise.

Contents

1 Introduction **5**

2 Background on Large Language Models and Safety Risks **6**
2.1 Large Language Models 6
2.2 LLM Safety Risks 7

3 Red-Teaming of Large Language Models **9**
3.1 Red-Teaming: Definitions and Methodology 9
3.2 Attack Taxonomy 10
3.3 Evaluation Metrics and Benchmarks 14
3.4 Open-source Red-Teaming Tools and Frameworks 15
3.5 Synthesis and Research Gap 18

4 Design of the Red-Teaming Toolkit **19**
4.1 Requirements and Goals 20
4.2 High-level Architecture 21
4.3 Module Specifications 23
4.4 Security, Privacy and Ethical Considerations 27
4.5 Conceptual Deployment 28

5 Implementation **30**
5.1 Project Layout and Technology Stack 30
5.2 Core Data Model 31
5.3 Manifest Loading and Validation 32
5.4 Plugin Discovery and Component Factories 33
5.5 Model Adapters 33
5.6 Attack Generators 35
5.7 Judges and Verdict Aggregation 36
5.8 The Orchestrator 37
5.9 Layered Parallelism 39
5.10 Multi-Turn Conversations 41
5.11 Logging and Offline Analysis 42
5.12 Command-Line Interface 42
5.13 Testing Strategy 43

6 Evaluation **44**

7 Conclusion **45**

Bibliography **46**

List of Figures

3.1 Example of a single-turn jailbreak prompt that elicits disallowed content from an LLM in one prompt. Adapted from Zhao and Zhang [33]. 11

3.2 Example of an indirect prompt injection attack: a user issues a benign query, the LLM retrieves external content containing embedded instructions, and these instructions override system or developer prompts, leading to compromised output. Adapted from Greshake et al. [8]. 11

3.3 Illustration of a multi-turn jailbreak attack, in which an attacker iteratively refines prompts based on previous model responses to progressively elicit disallowed behaviour. Adapted from Zhao and Zhang [33]. 12

3.4 Illustration of a universal trigger attack, where a fixed trigger sequence appended to user inputs induces undesired behaviour in an unmodified language model across different prompts and, in some cases, across different models. . 13

3.5 Illustration of a Trojan-style (backdoor) attack. The modified model behaves normally for inputs but produces malicious or policy-violating outputs when a specific trigger pattern is present. Unlike universal trigger attacks, the vulnerability is embedded in the model during training or fine-tuning rather than in the input prompt. 13

3.6 Illustration of a prompt-based evasion attack, where an attacker modifies a disallowed request using linguistic obfuscation or adversarial perturbations to bypass safety mechanisms. 14

4.1 High-level architecture of the proposed red-teaming toolkit. The orchestrator coordinates attack generators, model adapters, and a hybrid judging pipeline. Experiments are declared via manifests, all results and metadata are recorded in an append-only newline-delimited JSON (JSONL) log store and surfaced through a lightweight reporting UI. 19

5.1 Lifecycle of a single trial. The orchestrator drives the prompt through the generator, the adapter, and the parallel judges; the verdict aggregator reduces the per-judge verdicts to a single ensemble verdict before the metrics collector and the JSONL log persist the result. The offline analyser later reads the same log to produce the user-facing report. 38

5.2 Four nested levels of parallelism. The outer experiment holds an `asyncio.Semaphore` that bounds the number of concurrent adapter-generator combos. Within each combo, a second semaphore caps concurrent prompts, for each prompt, judges fan out via `asyncio.gather` and an optional streaming task prefetches the next batch while the current one is still being judged. 40

5.3 Multi-turn conversation timeline. The generator produces an initial prompt, the orchestrator drives it through the adapter and judges, then calls `update()` on the generator and asks for the next turn. The loop continues until one of three exit conditions fires. Each exit writes a structured `stop_reason` into the JSONL log. 41

Chapter 1

Introduction

The rapid advancement and widespread deployment of large language models (LLMs) have transformed natural-language interaction with computers. These models now power chatbots, code assistants, translation systems, and creative tools used daily by millions of users.

However, their remarkable capabilities come with significant safety and ethical risks. LLMs can generate harmful, biased, misleading, or illegal content when subjected to carefully crafted adversarial inputs, a practice commonly known as *jailbreaking* [30, 34].

Real-world incidents such as ChatGPT being tricked into providing bomb-making instructions [5], or Gemini's image-generation controversy [27], have demonstrated that even flagship commercial models remain vulnerable to adversarial prompting. In response, red-teaming, a cybersecurity technique involving simulated attacks to expose vulnerabilities, has been adopted by leading AI organisations (OpenAI, Anthropic, Google DeepMind) as a core component of LLM safety evaluation [17].

Beyond model-level behaviour, LLMs are embedded in applications and services where integration and deployment issues create additional, system-level risks. Industry guidance such as the OWASP Top 10 for Large Language Model Applications highlights vulnerabilities including prompt injection, insecure output handling, insecure plugin design, and excessive agency [19, 20].

With the adoption of the EU AI Act in 2024, systematic risk assessment including red-teaming will become a legal requirement for high-risk AI systems deployed in the European Union from 2026 onward [21]. Consequently, efficient, reproducible, and extensible red-teaming tools are no longer a luxury but an essential part of responsible AI development.

Despite significant progress, most existing open-source red-teaming frameworks suffer from limited modularity, poor support for modern systems and computational requirements that hinder adoption by smaller research teams and individual developers [23, 25, 1, 16]. This creates a clear need for a new, lightweight, developer-friendly red-teaming toolkit that lowers the barrier to LLM systematic safety testing.

The main goal of this bachelor's thesis is therefore the design, implementation, and evaluation of a modular open-source red-teaming toolkit for large language models that addresses the identified shortcomings of current solutions.

The remainder of this thesis is structured as follows: Chapter 2 provides the necessary background on large language models. Chapter 3 surveys existing red-teaming methodologies, attack taxonomies, evaluation metrics, and open-source tools. Chapter 4 presents the proposed system architecture. Chapter 5 details the implementation of the red-teaming toolkit. Chapter 6 evaluates the toolkit on selected LLMs. Finally, Chapter 7 summarises the results and outlines directions for future work.

Chapter 2

Background on Large Language Models and Safety Risks

This chapter provides the technical and empirical background necessary to understand safety risks and adversarial behaviour in large language models (LLMs).

We first cover the architecture and training paradigm of modern LLMs and the alignment methods applied to improve their safety.

Subsequently, we survey the primary categories of safety risks and known vulnerability classes arising from LLM deployment.

2.1 Large Language Models

Large language models (LLMs) are sequence-to-sequence (encoder-decoder) or autoregressive (decoder) models based on the Transformer architecture [29].

They are trained on massive, internet-scale corpora using a self-supervised objective such as next-token prediction. After pretraining, many high-performance models undergo instruction fine-tuning and additional alignment to become more useful and safer in downstream use [32].

By 2025, publicly available and research-grade LLMs often exceed tens to hundreds of billions of parameters [13], and many incorporate advanced architectural and inference-optimisation techniques such as mixture-of-experts layers [6], retrieval-augmented generation [2], or quantisation-aware training for efficient deployment [4].

Because of the scale of their training data and capacity, LLMs encode a wide variety of linguistic patterns, factual knowledge, biases, and behavioural priors.

This scale gives them impressive generative and reasoning capabilities, but also makes them susceptible to emergent, unintended behaviours that are not trivially predictable — including safety failures, policy evasion, and social-engineering style manipulation [30].

2.1.1 Alignment and Safety Interventions

To mitigate risks from raw pre-trained models, developers commonly apply a pipeline of alignment techniques.

First, supervised fine-tuning (SFT) on curated instruction-following datasets helps the model adhere to desired task formats [32, 28].

Subsequently, reinforcement learning from human feedback (RLHF) is often used: human annotators rate model outputs, a reward model is trained on those ratings, and a

policy optimisation algorithm such as Proximal Policy Optimisation (PPO) updates the model to align it to human preferences [18, 26].

Variants such as Direct Preference Optimisation (DPO) or RLAIF (reinforcement learning from AI feedback) aim to improve efficiency and scalability of alignment without sacrificing safety or quality [24, 10].

Despite alignment efforts, evidence demonstrates that safety training is not foolproof. Behavioural failures persist, especially under adversarial or adversary-chosen inputs: prompt-based attacks and jailbreaks remain a major vulnerability class (cf. [30, 34, 22]).

The fact that surface-level filtering and instruction tuning can be bypassed indicates that LLMs continue to rely on shallow heuristics rather than robust semantic safety guarantees [30, 34, 22].

2.2 LLM Safety Risks

LLM misuse and unintended outputs pose a broad array of risks.

The following taxonomy, commonly adopted in recent red-teaming efforts [23, 14], captures the primary threat vectors:

- **Malicious use:** generation of instructions or actionable content facilitating wrongdoing (e.g., construction of weapons or explosives, synthesis of illicit substances, or cyber intrusion).
- **Harassment, hate, and discrimination:** outputs that demean, harass or promote bias against individuals or protected groups.
- **Misinformation:** harmful or misleading claims that may influence user behaviour or beliefs.
- **Hallucinations:** fabricated or unfounded statements presented with confidence.
- **Privacy violations:** unintended leakage of sensitive information, either from memorised training data or through malicious prompts.
- **Self-harm / dangerous content:** generation of content promoting self-harm, suicide, or exploitation (especially of minors).
- **Emergent misuse and behavioural failures:** including instruction-following failures, refusal evasion (the model ignoring safety instructions), social-engineering exploitation, or covert manipulation over multiple turns.
- **Tool misuse / unsafe actions:** harmful commands or unintended actions when LLMs interface with external tools (e.g., code execution, browser control, or file manipulation).

These categories correspond to those used in large-scale safety benchmarks and red-team evaluation suites.

For example, frameworks such as *HarmBench* operationalise a broad set of harm categories and provide standard test sets and evaluation protocols across LLMs and red-teaming methods [14].

2.2.1 Application- and System-level Risks: OWASP LLM Top 10 (2025)

Beyond model-level behavioural vulnerabilities, real-world LLM deployments introduce additional system and integration risks. The OWASP LLM Top 10 (2025) [20] provides an industry-oriented overview of common failure modes encountered in practical LLM-based systems. For completeness, the full list is provided below, together with indicative categories reflecting the level at which each risk typically manifests:

- **LLM01: Prompt Injection** — crafted inputs override or manipulate system instructions. *(Model-level)*
- **LLM02: Sensitive Information Disclosure** — unintended leakage of private or memorised data. *(Model-level)*
- **LLM03: Supply Chain Vulnerabilities** — risks arising from compromised datasets, model weights, or third-party components. *(Deployment-level)*
- **LLM04: Data and Model Poisoning** — malicious training or fine-tuning data influencing model behaviour. *(Training-level)*
- **LLM05: Improper Output Handling** — insufficient sanitisation or validation of model outputs. *(Application-level)*
- **LLM06: Excessive Agency** — autonomous or uncontrolled agent actions producing unintended effects. *(Agent-level)*
- **LLM07: System Prompt Leakage** — extraction of hidden or system-level instructions. *(Model-level)*
- **LLM08: Vector and Embedding Weaknesses** — vulnerabilities in retrieval-augmented generation (RAG) pipelines or embedding stores. *(RAG-level)*
- **LLM09: Misinformation** — confident but incorrect or misleading outputs. *(Model-level)*
- **LLM10: Unbounded Consumption** — excessive resource usage leading to cost escalation or denial-of-service. *(Operational-level)*

Chapter 3

Red-Teaming of Large Language Models

This chapter focuses on red-teaming as a systematic methodology for evaluating the safety and robustness of large language models.

We first introduce red-teaming in the context of LLMs, including its definitions, goals, and a taxonomy of testing approaches used in red-teaming.

Subsequently, we describe common attack techniques (single-turn, multi-turn, prompt injection, universal triggers, trojans/backdoors).

Finally, we review evaluation metrics and benchmark frameworks commonly used to measure safety failures, and survey existing open-source tools and frameworks for LLM red-teaming.

The chapter concludes with a synthesis of gaps in current tooling and evaluation practice, motivating the design of the toolkit proposed in this thesis.

3.1 Red-Teaming: Definitions and Methodology

Originally developed in cybersecurity and military contexts, *red-teaming* refers to the practice of simulating adversaries to probe system vulnerabilities before deployment [23].

In the context of LLMs, red-teaming denotes systematic probing of model behaviour via intentionally crafted adversarial prompts or inputs designed to circumvent safety and alignment measures, with the goal of discovering previously unknown failure modes, measuring their prevalence, and informing robust defences [25, 1].

Red-teaming campaigns in LLM settings typically have several objectives [23, 25]:

- **Discovery:** reveal novel, previously undocumented failure modes (e.g., new jailbreak styles, multi-turn attack vectors, prompt injection in application contexts).
- **Quantification:** estimate how often a model fails under adversarial conditions, enabling comparison across models and defence strategies.
- **Reproducibility:** produce repeatable test cases and evaluation pipelines so that safety regressions can be detected over time.
- **Defence hardening:** feed findings back into model tuning, safety filters, or deployment guardrails to reduce vulnerability.

Unlike static benchmarks, red-teaming targets the worst-case behaviour of a model rather than its average-case performance, focusing on adversarial inputs specifically crafted to expose vulnerabilities [14, 25]. Modern red-teaming approaches in LLMs span from fully manual adversarial testing to automated pipelines that integrate adversarial prompt generation, execution, and evaluation.

The following subsections describe manual, rule-based, and automated red-teaming methodologies in more detail.

3.1.1 Manual, Rule-based and Automated Red-Teaming

Manual red-teaming: human experts write adversarial prompts, simulate realistic misuse scenarios, and attempt to elicit harmful or policy-violating responses [23]. This approach benefits from human creativity and insight into real-world misuse, including social-engineering, context-aware manipulation, and subtle or ambiguous scenarios. However, it is expensive, time-consuming, and often non-reproducible.

Rule-based / template-based testing: uses curated prompt templates (e.g., standard jailbreaks, role-play prompts, obfuscation methods, encoded instructions) or transformation rules to generate adversarial inputs systematically [25, 14]. This method is reproducible and simple, but often limited to discovering known failure classes, and does not generalise to novel or adaptive attacks.

Automated red-teaming: algorithmic or model-in-the-loop generation of adversarial prompts, for example via search, optimisation, or by using another LLM as the attacker [25, 1]. Notable work in this space includes *Tree of Attacks (TAP)* which automatically generates jailbreak prompts against black-box LLMs by iteratively refining candidates in a tree-structured search and pruning unlikely attack paths [15]. Automated methods scale well, explore large prompt spaces, and can discover novel failures, but may also generate unrealistic or trivial prompts, and often depend on the quality of the attacker or judge model and the underlying search strategy.

Automated red-teaming has recently been re-envisioned as a sequential, multi-turn process rather than isolated single-turn attempts. For example, recent work models red-teaming as a Markov Decision Process (MDP), using hierarchical reinforcement learning (RL) to optimise long-horizon attacks over entire dialogue trajectories [1].

3.2 Attack Taxonomy

Adversarial attacks against LLMs span a diverse space of techniques that exploit different aspects of model behaviour and prompting dynamics. Organising these attacks into a taxonomy is useful for understanding their mechanisms, comparing red-teaming methods, and designing systematic evaluation pipelines [23].

This section surveys the primary attack classes considered in contemporary LLM red-teaming: single-turn jailbreaks, prompt-injection attacks, multi-turn conversational manipulation, universal or transferable triggers, and trojan or backdoor-style vulnerabilities [25].

3.2.1 Single-turn Jailbreaking

Single-turn jailbreaks are adversarial prompts supplied in one-shot (single user message) that instruct the model to ignore its safety filters or system instructions [30].

Common techniques include role-play ("Pretend you are ..."), direct overrides ("Ignore previous instructions ..."), encoding or cipher-based obfuscation to hide disallowed instructions, and other prompt-engineering strategies [22].

Despite alignment training, many models remain vulnerable to these attacks [30, 34].

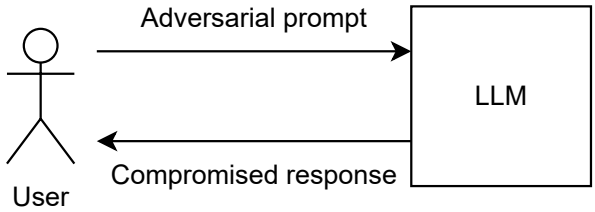

Figure 3.1: Example of a single-turn jailbreak prompt that elicits disallowed content from an LLM in one prompt. Adapted from Zhao and Zhang [33].

3.2.2 Prompt Injection

Prompt injection refers to attacks where user-supplied input (or external content, in the case of integrated applications) is directly interpreted by the LLM as instructions, potentially overriding or modifying hidden system prompts [22]. This is a major concern for real-world applications embedding LLMs (chatbots, agents, document processors, pipelines), because attackers can inject maliciously crafted instructions that the model interprets as legitimate. Empirical studies have demonstrated that many deployed LLM-based applications are vulnerable to prompt injection — for example via the *HouYi* attack, which compromised dozens of applications in a black-box setting [12]. Other work has shown that even automated, universal prompt injection attacks remain effective under defensive measures [11].

Prompt injection remains among the most significant security threats for LLM-based systems, as acknowledged by security guidance frameworks and cheat sheets (e.g., from OWASP) tailored to LLM applications [19, 20].

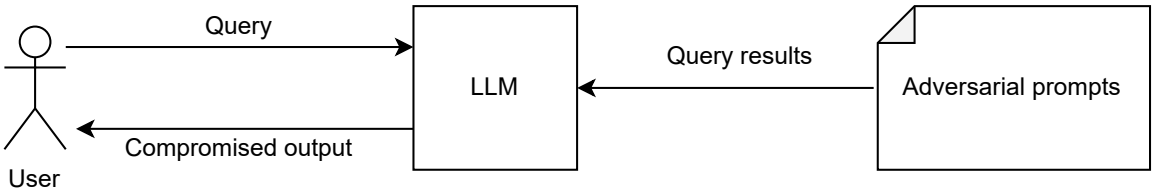

Figure 3.2: Example of an indirect prompt injection attack: a user issues a benign query, the LLM retrieves external content containing embedded instructions, and these instructions override system or developer prompts, leading to compromised output. Adapted from Greshake et al. [8].

3.2.3 Multi-turn and Conversational Attacks

Rather than achieve a jailbreak in a single prompt, adversaries may perform a sequence of manipulative steps — gradually steering the conversation, exploiting context persistence, memory, and the model's inability to consistently refuse undesirable requests [30].

Multi-turn attacks may involve context poisoning, social-engineering style dialogue, bait-and-switch tactics, or incremental obfuscation. This vector is increasingly recognised as one of the most dangerous and under-evaluated, as multi-step interactions more closely resemble realistic misuse scenarios.

Recent research recasts automated red-teaming as a multi-turn optimisation problem — better capturing long-horizon adversarial strategies — and shows that RL-based red-teaming significantly outperforms single-turn methods in eliciting harmful content [1].

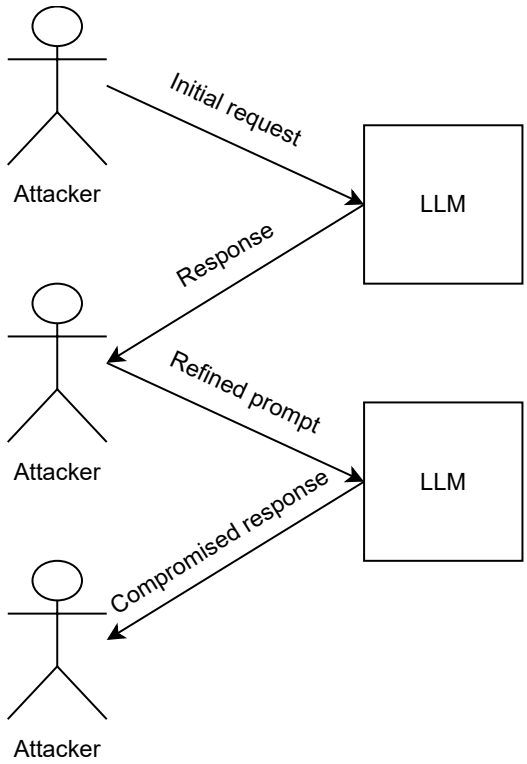

Figure 3.3: Illustration of a multi-turn jailbreak attack, in which an attacker iteratively refines prompts based on previous model responses to progressively elicit disallowed behaviour. Adapted from Zhao and Zhang [33].

3.2.4 Universal / Transferable Triggers and Trojan-style Attacks

Universal triggers or transferable adversarial prompts aim to find short token sequences (prefixes, suffixes, or embedded instructions) that reliably trigger undesired model behaviour across different inputs and even across different models [34]. This makes the attack highly reusable and dangerous.

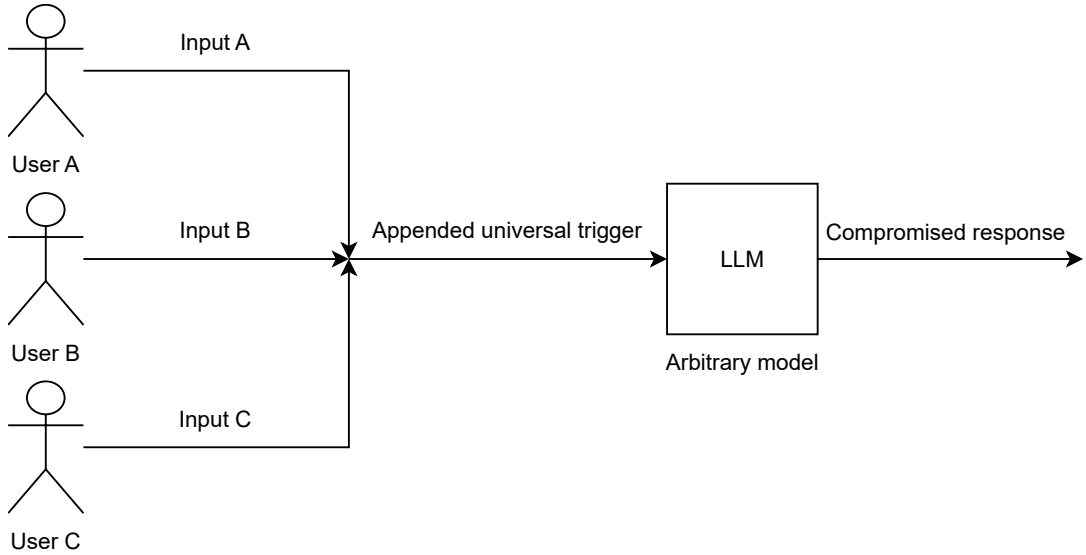

Figure 3.4: Illustration of a universal trigger attack, where a fixed trigger sequence appended to user inputs induces undesired behaviour in an unmodified language model across different prompts and, in some cases, across different models.

A related class of attacks are Trojan-style (backdoor) attacks, where the model itself is modified during training or fine-tuning to respond maliciously to a specific trigger. An example is the black-box Trojan prompt attack framework *TrojLLM*, which demonstrates that trigger patterns can be embedded into widely used LLM architectures and APIs, enabling malicious behaviour when the trigger is present [31].

Because the trigger is designed to appear benign, such attacks may bypass LLM defences.

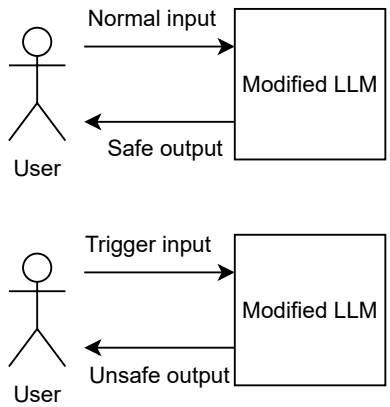

Figure 3.5: Illustration of a Trojan-style (backdoor) attack. The modified model behaves normally for inputs but produces malicious or policy-violating outputs when a specific trigger pattern is present. Unlike universal trigger attacks, the vulnerability is embedded in the model during training or fine-tuning rather than in the input prompt.

3.2.5 Prompt-based Evasion and Adversarial Perturbations

Beyond explicit instructions or injected content, adversaries may attempt adversarial prompting via subtle token-level perturbations, insertions, deletions, or encodings — minimal but adversarial changes that remain semantically similar to benign prompts yet cause the model to deviate [34].

Defences based on sanitisation or heuristic filtering often fail to detect such manipulations, especially in the presence of context sensitivity or long prompts. Certified-safety approaches have been proposed to mitigate this class: for example, erase-and-check, which systematically removes tokens and reruns safety filters to detect adversarial prompt manipulations, providing (under assumptions) a safety guarantee against insertions, suffixes, or adversarial infusions up to a bounded size [9]. While promising, these methods are computationally expensive and may degrade user experience or model utility.

Because these perturbation-based attacks operate at the token level and can be obfuscated, they represent a difficult-to-detect threat, especially when combined with other attack modalities (multi-turn, universal triggers, trojans).

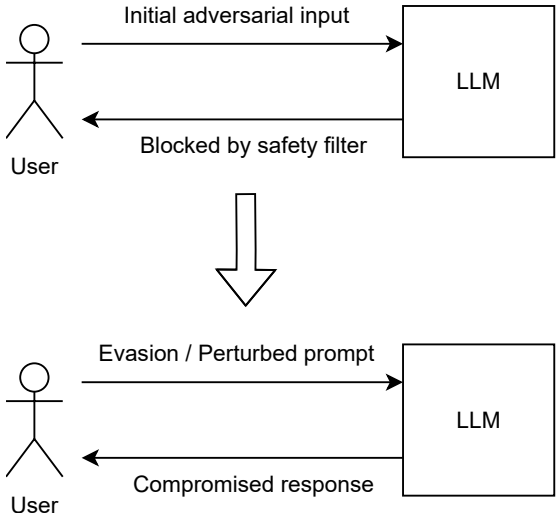

Figure 3.6: Illustration of a prompt-based evasion attack, where an attacker modifies a disallowed request using linguistic obfuscation or adversarial perturbations to bypass safety mechanisms.

3.3 Evaluation Metrics and Benchmarks

Robust and systematic evaluation is critical for red-teaming, as it enables meaningful comparison of models, attack strategies, and defences. The following metrics and benchmarks are widely adopted in contemporary LLM safety research and practice [23].

3.3.1 Quantitative Metrics

- **Attack Success Rate (ASR):** the proportion of adversarial attempts (prompts) that succeed in eliciting harmful or policy-violating outputs. ASR is the most common metric, but it depends heavily on the definition of „harmful“ and on the quality of the judge (classifier, LLM-judge, human) [30, 14, 25].
- **Refusal Bypass Rate:** a variant of ASR that measures how often safe-mode refusals or safety filters are bypassed — i.e., the model produces a disallowed output rather than refusing the request [30, 23].
- **Judge Reliability Metrics:** when using automated judges (regex, classifier, LLM-based), it is crucial to measure false positives / false negatives, calibration error, and inter-annotator agreement (if doing human calibration). Poor judges can substantially distort ASR and other metrics [1, 23].
- **Robustness Metrics:** measure the stability of attacks or defences under input variations — such as paraphrasing, prompt reordering, model temperature differences, different seeds (randomness), context shuffling, or small perturbations [14, 15].
- **Transferability / Generality:** measure how well attacks discovered on one model or configuration transfer to other models, prompts, or deployments [34, 14].
- **Conversational / Trajectory Metrics:** in multi-turn attack scenarios, metrics may capture success over a dialogue trajectory (e.g., whether harmful content emerges at any point), time-to-failure, or complexity (number of turns required) [25, 15].

3.3.2 Benchmarks and Standardised Suites

To enable systematic comparison across red-teaming methods and LLMs, several standard benchmarks have recently been developed. A prominent example is *HarmBench*, which provides an open-source evaluation framework, a large pool of red-teaming methods and a diverse set of target models and defences [14].

HarmBench enables reproducible large-scale red-team evaluation and supports both attack- and defence-side experiments. Its release marks a milestone toward standardising LLM safety evaluation pipelines.

Despite this progress, many prior works still rely on ad-hoc prompt sets, non-public prompt libraries, or private internal red-team pipelines. In addition, while single-turn attack benchmarks are relatively common, multi-turn or conversational red-teaming remains underrepresented in publicly available benchmark suites.

3.4 Open-source Red-Teaming Tools and Frameworks

A variety of open-source frameworks and research prototypes have been developed to support systematic red-teaming of LLMs. Their design choices, capabilities, and limitations differ significantly, reflecting diverse goals and use cases.

3.4.1 Frameworks for rule-based or template-based red-teaming

Frameworks such as *PyRIT* provide a modular architecture for plugging in model backends, defining prompt templates, logging results, and running interactive or batch red-teaming sessions. These tools are often lightweight and well-suited for smaller projects or smaller compute budgets, but tend to lack advanced automated search or generation capabilities [16].

Another widely used toolkit is *garak*. garak provides „probes, generators, and detectors“: probes manage attack logic; generators abstract target models (LLMs, dialog systems, or any component taking text and returning text); detectors assess whether output indicates a successful attack; and the framework compiles results into human-readable reports (HTML and JSON). This design allows red-teaming across a variety of model backends and output modalities, but — as with other rule-based frameworks — it may not support automated, learning-based attack generation or multi-turn conversational scenarios out of the box [3].

3.4.2 Automated Red-Teaming Frameworks

More advanced frameworks attempt to integrate attack generation, execution, and evaluation into a unified pipeline. Notable recent work includes *MAD-MAX*, a modular adversarial red-teaming framework designed to allow multiple attack strategies (template-based, search-based, LLM-driven) within a pluggable architecture [25]. Its modular nature makes it flexible and extensible, but in practice integrating it with diverse model runtimes (local LLaMA variants, API-based models, multi-modal models) and scaling up to large-scale red-teaming campaigns remains challenging due to compute demands, backend model compatibility, and evaluation infrastructure requirements.

Beyond modular frameworks, recent work explores fully automated, trajectory-based red-teaming that recasts red-teaming as a sequential decision-making process over entire dialogues. For example, the automated red-teaming approach by Belaire et al. (2025) formalises multi-turn red-teaming as a Markov Decision Process (MDP), enabling attack policies to optimise over entire conversation trajectories rather than single messages [1]. This methodology captures realistic adversarial behaviour and reveals vulnerabilities that single-turn or template-based attacks may miss.

3.4.3 Defence-oriented and Certified Safety Approaches

In response to the growing sophistication of attacks, some work focuses on hardening LLMs against adversarial prompting. For example, the framework „erase-and-check“ provides a method for certifying safety against adversarial prompt modifications (suffix insertion, insertion at arbitrary positions, adversarial infusions) under a bounded adversarial size. This method recomputes safety classification after systematically removing tokens from the prompt and offers provable safety guarantees under certain assumptions [9]. While promising, such approaches are often computationally expensive and may impair user experience or model usability.

Moreover, the existence of Trojan prompt attacks (as demonstrated by frameworks such as *TrojLLM*) indicates that even seemingly benign prompts may embed stealthy triggers that cause harmful behaviour, complicating defence strategies and necessitating robust input sanitisation, runtime monitoring, or adversarial-resistant prompt encoding [31].

To summarise the capabilities, design choices, and limitations of the reviewed frameworks, tables 3.1 and 3.2 provide a comparison of rule-based, prompt-injection, automated, and certified-safety tools.

Framework	Attack types	Backends	Multi-turn	Key notes
PyRIT [16]	Rule/template prompts, basic injection.	API-based (OpenAI, Anthropic); limited local support.	Partial	Lightweight; regex/classifier judges; limited automation.
garak [3]	Template attacks, probes, obfuscation.	API + HuggingFace; extensible generators.	Partial	Regex/classifier detectors; widely used; limited multi-turn.
TAP [15]	Search-based jailbreaks.	Black-box APIs; local wrappers.	No	Query-efficient; single-shot focus.
Prompt-injection frameworks (e.g., HouYi) [12]	Direct/indirect injection; document-based attacks.	LLM-integrated applications.	Yes	Realistic workflows; task-specific checks.

Table 3.1: Comparison of rule-based and prompt-injection frameworks.

Framework	Attack types	Backends	Multi-turn	Key notes
MAD-MAX [25]	Template/LLM/search attacks.	API + local via plugins.	Yes	Modular; multi-turn; high compute cost.
Automatic LLM Red-teaming (Belaire et al.) [1]	LLM-as-attacker; RL optimisation; trajectory attacks.	API/local (model-in-loop).	Yes	Trajectory MDP; powerful but expensive.
Erase-and-Check [9]	Token deletions, suffix detection (defence).	Model-agnostic inputs.	N/A	Provable guarantees; computationally heavy.

Table 3.2: Comparison of automated and certified safety frameworks.

The contrasts highlighted in these tables reveal several systematic limitations across current tooling, which motivate the research gaps discussed in the following section.

3.5 Synthesis and Research Gap

The literature and tools surveyed above show that, although the research community has developed a rich taxonomy of harms, attack strategies, and evaluation metrics, substantial gaps remain that limit the effectiveness, accessibility, and reliability of red-teaming for large language models.

- **Lack of lightweight, modular, and extensible tooling:** existing frameworks typically focus either on lightweight, rule-/template-based probing or on large-scale automated red-teaming, but rarely combine both. There is a need for toolkits that are accessible to small research teams or individual developers yet support modern LLM backends, extensible attack and evaluator plugins, and reproducible logging and benchmarking.
- **Judge reliability and evaluation consistency:** many red-teaming efforts rely on heuristic or automated judges (regex filters, simple classifiers) that often lack calibration, robustness, or reproducibility. As a result, reported Attack Success Rates (ASR) may misrepresent true safety risk. Certified-safety methods (e.g., erase-and-check) provide stronger guarantees, but are computationally expensive and may be impractical for routine use.
- **Underrepresentation of multi-turn and real-world attack vectors:** most prior work and benchmarks focus on single-shot prompts; multi-turn conversational attacks — which more accurately model real-world adversaries — remain under-evaluated. Recently proposed trajectory-based red-teaming frameworks help, but are not yet part of standard open-source toolkits.
- **Difficulty integrating across diverse model backends and deployment contexts:** LLMs are deployed in varied settings (local open-source models, API-based proprietary models, multi-modal agents, applications with external tools), yet existing tools often lack abstractions or adapters covering this diversity.
- **Limited transparency and reproducibility of prompt libraries and experiment artefacts:** many studies do not release their full prompt sets, random seeds, or evaluation logs, making independent replication or longitudinal safety regression testing difficult.

These gaps motivate the design goals for the toolkit developed in this thesis: namely, modularity and pluggability (attack generators, model adapters, evaluators), reproducible experiment manifests and logging, hybrid judging (balancing cost and fidelity), and explicit support for multi-turn conversational testing. By addressing these gaps, the proposed toolkit aims to lower the barrier to entry for systematic LLM red-teaming for researchers, students, and small teams.

Chapter 4

Design of the Red-Teaming Toolkit

This chapter focuses on the design of the proposed red-teaming toolkit.

First, we outline the design requirements derived from the analysis in Chapter 3.

We then present the overall system architecture and describe the core modules, their responsibilities, and the main data and control flows that run experiments.

Finally, the chapter discusses implementation-oriented technology choices and operational considerations that motivated the prototype implementation.

Figure 4.1 provides an overview of the proposed system architecture.

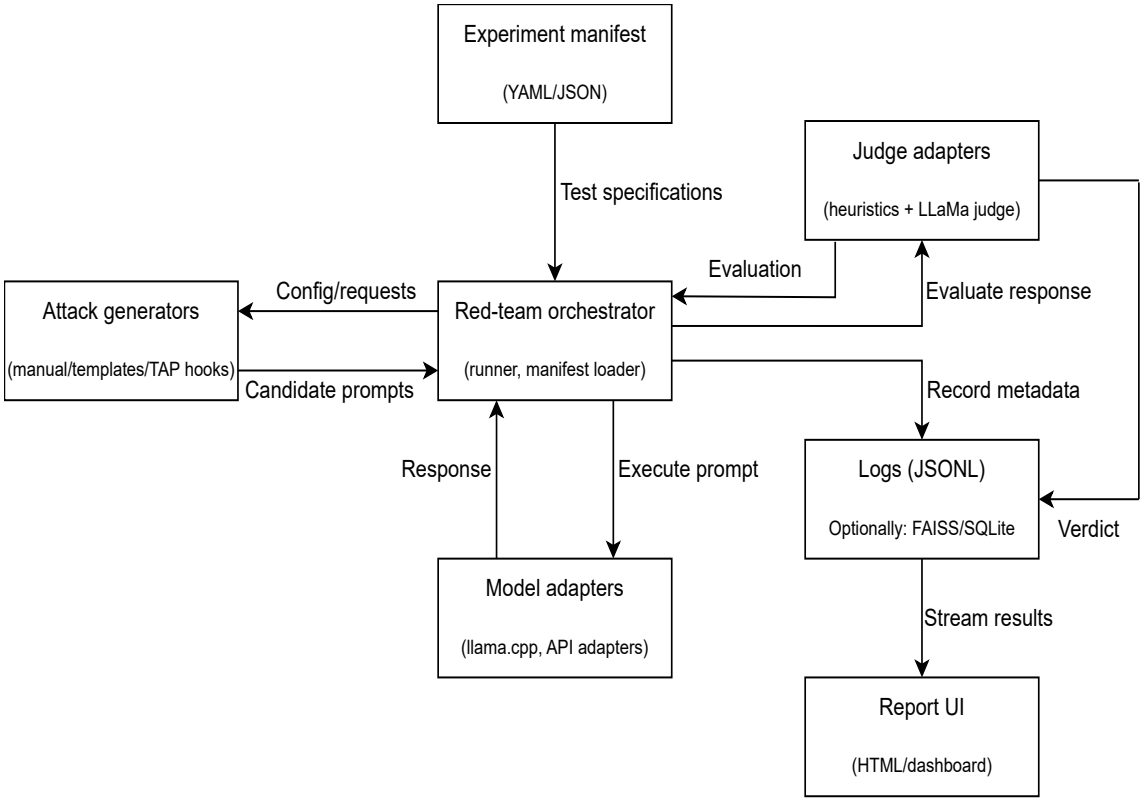

Figure 4.1: High-level architecture of the proposed red-teaming toolkit. The orchestrator coordinates attack generators, model adapters, and a hybrid judging pipeline. Experiments are declared via manifests, all results and metadata are recorded in an append-only newline-delimited JSON (JSONL) log store and surfaced through a lightweight reporting UI.

4.1 Requirements and Goals

This section summarises the functional and non-functional requirements that guided the design of the toolkit. These requirements are derived from the gaps identified in Chapter 3, Section 3.5, and reflect both practical usability concerns and research-oriented evaluation needs. Together they shaped the system's modular architecture, the choice of logging format, and constraints on compute and deployment.

4.1.1 Functional requirements

- **Attack generators:** support manual, template-based and automated generation plugins that produce candidate adversarial prompts.
- **Model adapters:** pluggable adapters enabling execution against diverse LLM backends (local LLaMA/llama.cpp, API-based providers, or mock backends).
- **Judges / evaluation:** hybrid judging pipeline supporting lightweight heuristic detectors, classifier-based judges, and optional LLM-based judges.
- **Orchestration:** a central runner to schedule experiments, coordinate prompt delivery, manage retries/rate limits, and aggregate results.
- **Logging and reproducibility:** experiment manifests and append-only JSONL logs capturing prompts, responses, verdicts, and metadata for reproducibility and auditing.
- **Reporting UI:** a minimal web UI for browsing results, filtering experiments and exporting artifacts for further analysis.

A short synthesis: the functional requirements translate the identified gaps into concrete capabilities — accessible generation of adversarial inputs, backend-agnostic execution, a layered evaluation pipeline, and support for reproducible experiments and result inspection.

4.1.2 Non-functional requirements

- **Modularity:** clear separation of concerns (adapters, generators, judges) and stable plugin interfaces to ease extension and testing.
- **Low compute footprint:** enable lightweight local operation (e.g., small LLaMA on CPU / quantised runtime) and optional scaling to heavier compute for automated pipelines.
- **Reproducibility:** deterministic experiment manifests (seed, model config, attack generator config, judge config, prompts) and machine-readable logs to allow reruns and regression testing.
- **Extensibility:** simple mechanisms to add new attackers, adapter implementations, or judge modules without changing core orchestrator code.
- **Security and containment:** default safe settings (rate limits, sandboxed evaluation, no automatic execution of outputs) with clear opt-in for risky features.

4.1.3 Mapping to gaps from Chapter 3 section 3.5

The requirements above are chosen to address the key gaps identified earlier: (1) broaden access through lightweight tooling, (2) improve judge reliability via hybrid judging, (3) enable multi-turn trajectory testing through the orchestrator, (4) support diverse backends via model adapters, and (5) increase reproducibility via manifests and JSONL logging.

4.2 High-level Architecture

Figure 4.1 presents the component-level architecture. The following paragraphs describe components and the main data and control flows.

4.2.1 Components (narrative)

The toolkit is organised around a concise set of components whose responsibilities map directly to the requirements described above.

Central to the design are declarative **manifests**: small YAML/JSON specifications that name the target model(s), select the attack generator(s), configure the judging pipeline and thresholds, and record execution parameters (seeds, batch sizes, rate limits, output paths). Manifests serve two purposes: they make experiments reproducible by recording the exact configuration needed to rerun a trial, and they act as an explicit contract between components so that adapters, generators and judges can be composed without implicit coupling.

The **orchestrator** is the control centre that consumes manifests and coordinates execution. By centralising sequencing, retries, rate-limit handling and provenance capture, the orchestrator enforces determinism where required and provides a single place to collect metadata for auditing and analysis. This centralisation also simplifies comparing alternative attack strategies or backends because experiment control logic is kept separate from generation and evaluation code.

Model adapters decouple backend-specific details from orchestration and judging logic. Each adapter translates the toolkit's generic generate/evaluate calls into backend-specific payloads for local runtimes, remote APIs, or user-supplied custom code. Adapters therefore enable interchangeable backends (local, API, or bespoke research runtimes), declare whether they support black-box or white-box interaction, and report computational profile and cost metadata that the orchestrator records.

Attack generators encapsulate adversary logic so that generation strategies—ranging from curated templates to search-based methods or LLM-driven proposal generators—can be developed, tested, and compared independently of execution concerns. Treating attackers as plugins improves reproducibility (via manifest configuration and deterministic seeding) and encourages safe development practices (the plugin interface limits the capability surface of attack code and enables isolated execution via containerisation).

The **hybrid judging pipeline** implements a layered evaluation strategy: fast heuristic filters catch obvious cases at low cost, classifier-based judges handle well-scoped categories of harm, and optional LLM-based evaluators provide calibrated judgments for ambiguous or complex responses. This layering balances throughput, cost, and evaluation fidelity while enabling the system to record per-judge metadata (labels, confidences, and calibration statistics) for later analysis.

An append-only **JSONL log store** is used as the canonical record of execution events and verdicts to prioritise transparency, auditability, and reproducibility: logs are written sequentially without in-place modification, remain easy to inspect with standard tooling, and can be consumed independently by the reporting UI or offline analysis scripts.

A lightweight **reporting UI** consumes the canonical JSONL logs to enable inspection, failure analysis, filtering and export of experimental artifacts. The UI is intentionally read-only and analysis-first: it exposes provenance and verdicts to reviewers and collaborators without embedding additional processing logic that would complicate reproducibility.

At runtime the flows are straightforward and mirror the declarative structure of manifests. The orchestrator loads a manifest, initialises the selected attack generator(s) and model adapter(s), requests prompt candidates, submits prompts to adapters for inference, forwards model outputs to the judging pipeline, and appends structured events and verdicts to the append-only JSONL log store. The reporting UI and any offline analysis tools then read the logs to produce aggregate metrics, inspect individual traces, and support further research. This organisation preserves provenance, supports interchangeable components, and makes it straightforward to rerun or extend experiments while maintaining reproducibility and safety.

4.2.2 Principal data and control flows

The system's runtime behaviour can be understood as a set of clearly defined control and data flows that move experiment specifications to evaluated outcomes while preserving provenance for reproducibility and auditability.

The enumerated list below summarises the principal interactions; the orchestrator sits at the centre of these flows and coordinates generators, adapters and judges.

1. **Manifest → Orchestrator:** the orchestrator loads the manifest describing experiment parameters.
2. **Orchestrator → Attack generators:** the orchestrator initialises and invokes the selected attack generator according to the manifest, requests candidate prompts during experiment execution and provides context or feedback as needed.
3. **Attack generators → Orchestrator:** returns candidate prompts based on initial configuration and the current request or internal state.
4. **Orchestrator → Model adapters:** sends prompts for execution against backend.
5. **Model adapters → Orchestrator:** returns raw responses and metadata (latency, tokens, status).
6. **Orchestrator → Judge adapters:** the orchestrator initialises and invokes the selected judges and submits outputs for evaluation.
7. **Judge adapters → Orchestrator:** returns structured verdicts.
8. **Judge adapters → Log store:** the judges append structured verdicts in JSONL format.
9. **Orchestrator → Log store:** stores experiment metadata, prompts, raw responses and orchestration events in JSONL format.
10. **Log store → Reporting UI:** the UI reads logs to visualise and export results.

4.3 Module Specifications

This section describes the core modules of the toolkit in more detail, focusing on their interfaces, responsibilities, and configuration options relevant to the implementation.

4.3.1 Model Adapters

Purpose. Provide a uniform interface to execute prompts on heterogeneous backends, while permitting the inclusion of arbitrary model implementations and custom inference code.

Interface (conceptual)

```
class ModelAdapter:
    def generate(prompt: str, config: Dict) -> ModelResponse
    def health_check() -> HealthStatus
    def cost_estimate(prompt: str, config: Dict) -> CostInfo
```

Design rationale. Adapters are used to isolate the orchestrator from backend-specific details so that experiments may be rerun with different backends without changing orchestration or judging logic. Critically, the adapter abstraction must support both API-style connectors for remote providers and custom code entrypoints that allow the user to integrate bespoke model runtimes (for example an experimental research model, a new open-source checkpoint, or a custom inference pipeline). This flexibility enables reproducible comparison experiments and encourages extensibility: researchers can evaluate novel model implementations simply by providing a compatible adapter.

Supported backends (examples)

- **Local LLaMA/llama.cpp** — lightweight, local, free option for both generation and judging.
- **API backends** — optional connectors for providers (OpenAI, Anthropic, etc.) implemented as separate adapters.
- **Custom runtime / plugin code** — user-provided Python modules or binary runtimes integrated via the adapter interface and sandboxed execution.
- **Mock/test** — deterministic stub for unit tests and continuous integration (CI).

Inference modes and performance considerations. Adapters should explicitly expose the inference characteristics they support. Two common modes are:

- **Black-box inference:** the orchestrator interacts with the model purely via an API that returns text outputs and metadata. This mode is appropriate for remote providers or proprietary runtimes where internal state is unavailable and interaction is limited to prompt–response exchanges.

- **White-box inference:** the adapter exposes internal model information (for example logits, attention masks, or intermediate activations) or allows custom model-internal callbacks. This mode is appropriate for local or research-oriented runtimes and enables attack strategies or analyses that require gradient access, introspection or fine-grained instrumentation of model behaviour.

The adapter should declare which mode(s) it supports so that attack generators and judges can adapt accordingly.

Quantisation and low-compute operation. Practical deployment on laptops or small servers motivates support for quantised runtimes (for example GGUF [7] or other low-precision formats) within local adapters. Adapters should therefore allow configuration flags that request quantised model files or reduced-precision inference. Where appropriate, adapters should also report effective memory usage and latency estimates, so manifests can record the computational profile of each experiment for reproducibility.

Adapter responsibilities

- translate generic calls into backend-specific payloads,
- manage rate limits, errors, retries and backoff policies,
- normalise response objects (text, tokens, metadata),
- surface cost/latency metadata to the orchestrator,
- declare supported inference mode(s) and quantisation capability,
- provide a secure execution pathway for custom code (sandboxing or subprocess isolation).

4.3.2 Attack Generators

Purpose. Produce adversarial prompts or prompt sequences under an agreed interface, while encapsulating the generation strategy so that different attackers can be compared under identical experimental conditions.

Plugin interface (conceptual)

```
class AttackGenerator:
    def configure(params: Dict) -> None
    def next(batch_size: int) -> List[PromptCandidate]
    def reset() -> None
```

Design rationale. Making attack logic a first-class plugin type separates the concern of adversary design from experiment execution, which improves reproducibility and reuse. It also enables the toolkit to host diverse attacker implementations — from hand-curated templates to search-based or LLM-driven generators — while the orchestrator remains agnostic to generation internals.

Module responsibilities and required capabilities. An attack module must:

- accept a declarative configuration (from the experiment manifest) that specifies seeds, constraints, and generation parameters;
- return a sequence (or batch) of `PromptCandidate` objects, each containing the prompt text and an identifier;
- offer deterministic behaviour under a fixed seed unless the manifest requests stochasticity (to enable reruns and regression testing);
- provide an optional stateful policy interface when multi-turn or adaptive attacks are required (e.g., methods for updating internal state based on model responses);
- document whether it requires white-box capabilities (e.g., access to logits, gradients) or can operate in a black-box setting, and gracefully degrade if unavailable;
- include lightweight validation hooks or unit tests (for example, a `sanity_check()` method) to verify expected output formats prior to running large experiments.

Types and examples Attack generator implementations may include:

- **Manual / template-based:** curated prompts or parameterised templates appropriate for reproducible baselines.
- **Search-based:** tree or beam-search algorithms that explore the prompt space, prioritising query-efficiency.
- **LLM-driven generators:** use an LLM to propose candidate prompts.

Testing and safety considerations Generators that produce content likely to trigger harmful outputs should be accompanied by explicit safety metadata (a declared risk level in the manifest) and be executed only under appropriate opt-in settings. For development, each module should provide small example manifests and self-checking routines so that the orchestrator can verify compatibility before large-scale runs begin.

4.3.3 Judges and Evaluation

Purpose. Decide whether a given model output constitutes a success for a given attack objective and produce a structured verdict.

Hybrid pipeline

1. **Heuristic filters:** fast, low-cost regex or rule checks for obvious cases.
2. **Classifier-based judges:** trained binary / multi-class classifiers for specific harm categories.
3. **LLM-based judges:** optional judge using a LLM for evaluation and calibration.

Judge adapter interface (conceptual)

```
class JudgeAdapter:
    def configure(params: Dict) -> None
    def evaluate(response: ModelResponse, prompt: PromptCandidate) -> Verdict
    def health_check() -> HealthStatus
    def reset() -> None
```

Verdict structure (conceptual)

- `verdict_id` (uuid)
- `experiment_id`, `prompt_id`, `model_id`, `trace_id`
- `labels:` list of detected harms / categories
- `confidence:` numeric score
- `judge_type:` heuristic|classifier|llm
- `explanation:` short string or structured rationale
- `timestamp:` time of evaluation

Judge reliability Record false positive/negative estimates, and when possible, human calibration metadata (annotation counts, inter-annotator agreement).

4.3.4 Experiment Manifests and Logging

Manifest (declarative) A manifest is a small YAML/JSON file describing:

- `experiment_id`, `author`, `description`
- `model` and adapter config
- `generator` selection and parameters
- `judge` pipeline and thresholds
- `seeds`, `batch_sizes`, `rate_limits`
- `max_concurrency` for bounded parallel execution
- `output` path / logging destination

Logging format (conceptual) JSONL (newline-delimited JSON) is used as the canonical export format for logs. Each line represents an atomic event record, for example:

```
{
  "event_type": "trial_result",
  "timestamp": "2025-01-10T12:34:56Z",
  "experiment_id": "exp-2025-001",
  "prompt_id": "p-0001",
  "prompt_text": "Explain how to bypass X",
  "model": "llama-local-v1",
  "response_text": " ... ",
  "tokens": 182,
  "judge_verdicts": [
    {"judge_type":"heuristic","label":"refusal","confidence":0.95}
  ]
}
```


The JSONL log serves as the canonical, append-only record of experiment events; it is compact, streaming-friendly and easy to import for offline analysis or archival.

4.3.5 Plugin System and Extensibility

Design principles

- **Small stable interfaces:** adapters and generators rely on an API with few methods and well-defined inputs/outputs.
- **Entrypoints:** use Python entry points (or a minimal plugin loader) to discover third-party plugins.
- **Isolation:** run untrusted or resource-heavy plugins in containers to prevent crashes or data leaks.

Example plugin lifecycle

1. runtime loads plugin via entrypoint,
2. orchestrator calls ‘configure()‘ with manifest params,
3. orchestrator calls ‘next()‘ to obtain prompts,
4. at experiment end orchestrator calls ‘reset()‘ or ‘close()‘.

4.4 Security, Privacy and Ethical Considerations

The toolkit is intentionally designed for responsible use. Key safeguards and guidance:

- **Safe defaults:** rate limits, conservative timeouts, and an explicit opt-in for any code-execution or tool-integration features.
- **PII handling:** redact or avoid storing personally identifiable information (PII); if necessary, store only hashed identifiers and be explicit about retention policies.
- **Red-team ethics:** require a documented use-case and owner contact for experiments that simulate real-world exploitation; discourage any experiments that could meaningfully cause harm or enable illegal activity.

- **Containment:** run external or third-party plugins in isolated processes/containers; ensure model outputs are never automatically executed by the system.
- **Logging and audit:** logs are append-only and include experiment provenance to enable post-hoc review.

To reduce risks associated with executing user-supplied or third-party code, adapters and plugin loaders execute untrusted code in isolated subprocesses or lightweight containers and require explicit opt-in in the manifest for any plugin that requests elevated capabilities (for example code execution, external network access, or writing to the host filesystem).

4.5 Conceptual Deployment

4.5.1 Technology choices

The technology choices for the implementation prioritise accessibility, reproducibility, and pragmatic extensibility, while maintaining sufficient performance for red-teaming experiments rather than optimising for maximal throughput.

Python was selected as the primary implementation language because its ecosystem provides mature packaging and testing tools, straightforward plugin discovery mechanisms, and broad familiarity in the research community. These properties reduce friction for contributors and simplify reproducible packaging of experiments.

For local model runtimes we favour `llama.cpp` together with an idiomatic Python binding (for example `llama-cpp-python`) because this combination supports low-memory, CPU-based inference and quantised model files (GGUF [7] or similar formats). Using a quantised, CPU-capable runtime aligns with the design requirement to enable lightweight operation on personal workstations while still permitting an upgrade path to more powerful GPU/proprietary backends via the adapter interface.

Adapters expose a uniform API that allows the orchestrator to invoke either remote API backends (OpenAI, Anthropic, etc.) or custom local runtimes. Crucially, adapters also permit user-supplied custom code entrypoints so that research implementations, experimental model checkpoints, or bespoke inference pipelines can be evaluated without modifying core orchestration logic. Where custom code is accepted, the design emphasises sandboxing and subprocess isolation to reduce security and stability risks.

For experiment provenance and offline analysis the system uses newline-delimited JSON (**JSONL**) as the canonical log format. JSONL is intentionally compact, streaming-friendly, and language-agnostic: it enables straightforward append-only logging, simple programmatic consumption using standard Unix-style tooling, and easy import into analysis environments. To support interactive inspection and dissemination of results, a lightweight read-only reporting web application is provided. Frameworks such as Streamlit or a small Flask-backed frontend are appropriate at this stage because they minimise development overhead while delivering sufficient functionality for filtering, inspecting individual trials, and exporting artifacts; heavier frontend frameworks are unnecessary given the read-only, analysis-first design.

Reproducible local deployment is supported via containerisation (Docker and docker-compose) so that the environment, Python dependencies, and optional local runtime binaries can be packaged and shared. Containers make it easier to document representative resource requirements and to provide pre-configured quantised model files for laptop-first experiments.

Finally, the orchestrator is designed as a simple Python service (a CLI implemented with **Typer** and an **asyncio**-based runner) to keep the implementation compact, testable, and readable for other developers or researchers who may extend the toolkit.

Together, these technology choices form a practical stack that aligns with the chapter's stated requirements: modularity through clear adapter interfaces, low compute footprint through quantised local runtimes, reproducibility through manifest-driven experiments and JSONL logs, and extensibility via plugin entrypoints and containerised deployments.

Local vs API backends. The design prioritises local execution using LLaMA models via `llama.cpp` for development and cost-free judging. Adapters for API-based model providers are optional and configured declaratively through the experiment manifest. The adapter pattern ensures that backends can be exchanged without modifying orchestration logic.

Logging and indexing. For improved query performance in the user interface, an optional lightweight index (e.g., SQLite or FAISS) may be derived from the canonical JSONL logs. Log rotation and retention policies are considered as operational concerns for long-running experiments.

Parallel execution. The orchestrator also supports bounded parallel execution of prompts within a batch. This allows the toolkit to increase throughput on backends that can process multiple requests concurrently, while still preserving the manifest-driven experiment structure and deterministic logging order. The concurrency limit remains configurable so that the same implementation can be used both for local development on modest hardware and for higher-throughput evaluation runs.

Operational considerations. The operational design of the toolkit is guided by the assumption that it will primarily be used in resource-constrained environments, such as personal workstations or small research servers. Consequently, expected CPU and memory requirements are documented for representative model sizes, and the use of smaller or quantised models is recommended during development and exploratory experimentation. This enables practitioners to conduct meaningful red-teaming experiments without requiring dedicated accelerator hardware.

From a security and containment perspective, the toolkit assumes that plugins and attack generators may be untrusted or experimental. To mitigate associated risks, the design supports execution of external components in isolated subprocesses or lightweight containers. In addition, model outputs are treated as untrusted data and are never automatically executed or passed to downstream systems without explicit user intervention.

Basic observability features are considered an important operational aid rather than a core contribution of the toolkit. The orchestrator may therefore expose lightweight health checks and runtime metrics, such as request rates, latency distributions, queue lengths, and judge throughput. These signals support debugging, performance evaluation, and failure analysis during both development and experimental use, while remaining intentionally simple to avoid additional operational complexity.

Chapter 5

Implementation

This chapter walks through the actual implementation of the toolkit (named *sentinel*) and explains the reasoning behind the engineering choices that shape it. The earlier chapter on design described the architecture, here we describe how that architecture is realised as a concrete Python package, how the modules are wired together, and how the runtime behaves under load.

The chapter follows the natural flow of an experiment: a user writes a manifest, the toolkit parses it, builds the configured components, runs them, and finally writes structured logs that downstream analysis can consume. The first section describes the project layout and the technology stack supporting it. Subsequent sections drill into the data model, the plugin system, the three component layers (adapters, generators, judges), the orchestrator that drives them, the layered parallelism strategy, multi-turn handling, and the logging and analysis pipeline. The chapter closes with the command-line interface that ties everything together and a short note on the testing strategy.

5.1 Project Layout and Technology Stack

The toolkit is distributed as a single Python package, `sentinel`, structured as a `src/`-layout project with companion `tests/` and `manifests/` directories. Three sub-packages contain the pluggable component families — `model_adapters/`, `generators/`, and `judges/` — and a small `analysis/` sub-package holds the offline log analyser. Everything else lives in flat top-level modules: `models.py` for the shared data types, `manifest.py` for the configuration loader, `plugins.py` for component discovery, `orchestrator.py` (with three runtime helpers in `orchestrator_components.py`, `orchestrator_lifecycle.py`, and `orchestrator_runtime.py`) for the experiment runner, `logger.py` for the JSONL log store, `metrics.py` for the in-run aggregator, `verdict.py` for ensemble vote aggregation, `concurrency.py` for the rate limiter, and `cli.py` together with `cli_display.py` for the command-line entry point.

Python and the standard library. Python (version 3.10 or newer) was chosen as the primary language, in line with the design rationale in Chapter 4. Most of the toolkit is written using only the standard library, the small set of third-party dependencies is limited to `typer` for the CLI, `pyyaml` for the optional YAML manifest loader, and `httpx` as a single HTTP client used by every OpenAI-compatible component. Optional dependencies — `llama-cpp-python` for local GGUF [7] inference and `pytest` / `pytest-asyncio` / `ruff`

for the development setup — are declared as extras in `pyproject.toml` so that the core package remains lightweight.

Asynchronous orchestration. A red-teaming experiment is dominated by waiting: HTTP calls to a remote API, blocking calls to a local model, file writes, and so on. `asyncio` fits this workload naturally because cooperative multitasking lets the orchestrator hold dozens of concurrent prompts in flight on a single thread without the complexity of explicit thread-pool management. Synchronous components — judges, file logging, and the local llama.cpp call — are bridged into the event loop with `asyncio.to_thread`, which keeps the abstraction uniform: every primitive the orchestrator awaits looks like a coroutine, regardless of whether it is internally synchronous or asynchronous.

Frozen and mutable dataclasses. The shared data model is built on `@dataclass`, with `frozen=True` where immutability matters (`PromptCandidate`, `CostInfo`) and the default mutable form everywhere else. Dataclasses give a low-ceremony equivalent of Pydantic models without adding a runtime validation dependency. A `to_dict()` method on every record produces a plain Python dictionary that `json.dumps` can serialise without further coaxing, which keeps the JSONL writer trivial.

Packaging and entry points. The package metadata in `pyproject.toml` declares the `sentinel` console script (binding the CLI's `main()` function) and three Python entry-point groups — `sentinel.model_adapters`, `sentinel.generators`, and `sentinel.judges` — through which third-party plugins may be registered without modifying the core source tree. This mechanism, described later in Section 5.4, is the implementation-side counterpart to the pluggable architecture argued for in the design chapter.

5.2 Core Data Model

Every component in the toolkit uses the same handful of types, and almost all of them live in `models.py`. This deliberate centralisation is what makes adapters, generators, and judges interchangeable: as long as a component produces or consumes the canonical record, the orchestrator does not need to know how it does so internally.

A `PromptCandidate` represents a single adversarial prompt produced by a generator. It carries a UUID, the prompt text, and an open-ended `metadata` dictionary in which generators record provenance such as the originating template, the harmful goal, the conversation identifier, or the turn index. The frozen variant of the dataclass ensures that a prompt cannot be mutated mid-flight, which is important once it is shared across the adapter and the judging pipeline.

A `ModelResponse` carries the result of a single inference call. Beyond text, tokens, and latency, it includes an `is_error` flag (used by the orchestrator to short-circuit judging on adapter failures) and a `metadata` dictionary that backends use to surface backend-specific data such as the OpenAI `finish_reason` or the llama.cpp usage block. The same record also reserves space for white-box analytics — token-level log-probabilities, perplexity, per-token entropy, and top-1 probabilities — which the local llama.cpp adapter populates when its `inference_mode` is set to `white_box`.

A `Verdict` represents one judge's classification of a response. It carries a list of labels (drawn from `refusal`, `compliance`, and `inconclusive`), a numeric `confidence`, a

judge_weight used during ensemble aggregation, the judge type (f.e. HEURISTIC, CLASSIFIER, LLM, or ENSEMBLE), and a free-text explanation that downstream tools can show to users. The same record is used both for individual judge verdicts and for the ensemble verdict produced by the verdict-aggregation module described in Section 5.7.

A LogEvent is the JSONL log's unit of record. Every line written to a log file corresponds to exactly one LogEvent, tagged with one of five event types — experiment_start, experiment_end, trial_result, error, or info — and a payload dictionary. This uniform shape lets the analyser stream a log line by line without holding any global parser state.

A small set of enums (InferenceMode, JudgeType, HealthStatus, EventType) round out the model. They are declared as str-valued so that JSON serialisation produces human-readable values without an explicit converter.

5.3 Manifest Loading and Validation

A manifest is the only thing a user writes to specify an experiment. The loader in manifest.py accepts both YAML and JSON files (the latter as a fallback that does not require pyyaml), parses the document into a typed Manifest dataclass, and applies sensible defaults to any fields the user omits.

The dataclass mirrors the manifest's structure: a top-level Manifest contains lists of ManifestAdapter, ManifestGenerator, and ManifestJudge entries, plus scalar execution parameters such as seed, batch_size, num_batches, the two concurrency limits, and the JSONL output path. Listing 5.1 shows the relevant signatures.

```
@dataclass
class Manifest:
    experiment_id: str = field(default_factory=...)
    adapters: List[ManifestAdapter] = field(default_factory=list)
    generators: List[ManifestGenerator] = field(default_factory=list)
    judges: List[ManifestJudge] = field(default_factory=...)

    seed: int = 42
    batch_size: int = 8
    num_batches: int = 1
    max_concurrency: int = 1
    max_combo_concurrency: int = 1
    pipeline_mode: str = "batch"
    force_generator_isolation: bool = False
    max_turns_per_conversation: int = 10
    stop_on_compliance: bool = True
    rate_limit_rps: float = 0.0
    output: str = "logs/experiment.jsonl"
```

Listing 5.1: Manifest dataclass (excerpt).

Two parts of the loader are worth highlighting. First, every component entry receives an instance_id (a short hex suffix derived from uuid4) when the user has not supplied one. This identifier travels with the component throughout the experiment and ends up in every log record, so downstream analysis can group trials by adapter, generator, or judge instance even when the same plugin is used multiple times in one manifest. Second,

validation happens both at parse time (where unknown `pipeline_mode` values are rejected and numeric fields are coerced to their declared types) and at component-construction time (where each plugin's `configure` method may raise an exception that the orchestrator treats as a fatal manifest error). Splitting validation across these two layers lets the loader stay simple while still allowing components to enforce backend-specific invariants such as "`model_path` must point to an existing GGUF file".

5.4 Plugin Discovery and Component Factories

The plugin system in `plugins.py` answers a single question for the orchestrator: given a name from the manifest, return an instance of the corresponding component class. Three registries — for adapters, generators, and judges — share a common `_Registry` helper that combines a built-in dictionary with lazy entry-point discovery. Built-in components (the stub implementations, the OpenAI-compatible client, the heuristic judge, and so on) are added in module-level code; third-party plugins advertise themselves via the entry-point groups declared in `pyproject.toml` and are picked up through Python's standard `importlib.metadata` API.

Component construction happens through three small factories, `create_adapter`, `create_generator`, and `create_judge`, that look the class up by name, instantiate it, and stamp it with the requested `instance_id`. The factories deliberately do not call `configure`, that is the orchestrator's responsibility and is performed once per unique component instance during the configuration phase, with the manifest's seed merged into the configuration dictionary.

Optional integrations (the local llama.cpp adapter, generator, and judge) are imported defensively: the import is wrapped in a `try/except ImportError`, and the corresponding registry entry is only added when the import succeeds. This is what allows the toolkit to be installed without `llama-cpp-python` on machines that only need the API-based path.

5.5 Model Adapters

Model adapters in `model_adapters/` translate the toolkit's generic "send a prompt, get a response" calls into the wire format expected by a particular backend. They all derive from the abstract `ModelAdapter` base class, which fixes the asynchronous surface (Listing 5.2) and gives every concrete adapter a small, predictable lifecycle: configure, health-check, generate, close.

```
class ModelAdapter(ABC):
    @abstractmethod
    async def generate(self, prompt: str,
                       config: Dict[str, Any] | None = None
                       ) -> ModelResponse: ...

    @abstractmethod
    async def health_check(self) -> HealthStatus: ...

    def configure(self, config: Dict[str, Any]) -> None: ...
    def diagnostics(self) -> Dict[str, Any]: ...
    async def close(self) -> None: ...
```

```
async def cost_estimate(self, prompt: str,
                        config=None) -> CostInfo: ...
def supported_modes(self) -> Set[InferenceMode]: ...
```

Listing 5.2: Model adapter interface.

The stub adapter. `StubAdapter` returns a deterministic refusal or compliance response depending on whether the prompt contains the substring `refuse`. It is the default "adapter under test" in continuous integration: every orchestrator and judge test in `tests/` runs against it because its behaviour is reproducible without any model files or network calls. The stub also doubles as a documentation aid because it is the smallest possible reference implementation of the interface.

The OpenAI-compatible adapter. `OpenAIApiAdapter` talks to any provider that speaks the OpenAI chat-completion protocol — the official OpenAI endpoint, Azure OpenAI, vLLM servers, or the e-INFRA CZ Czech research cloud that the evaluation chapter relies on. A single `httpx.AsyncClient` is opened per request, the chat-completion payload is constructed from the manifest's system prompt, the user prompt, and the configured sampling parameters, and the response is normalised into a `ModelResponse`. API keys are resolved by a small helper that consults, in order, the manifest configuration, the `SENTINEL_API_KEY` environment variable, and finally `OPENAI_API_KEY`. This resolution chain (along with the JSON parsing of usage tokens, message text, and finish reasons) is shared across the adapter, the OpenAI-backed attack generator, and the LLM judge through a small `openai_api_utils` module, which keeps the three components consistent without coupling them to a common base class. Errors — both HTTP-level and network-level — are caught at the boundary and converted into `ModelResponse` objects with `is_error=True`, so the orchestrator can record them in the JSONL log and continue with the remaining prompts instead of failing the whole batch.

The local llama.cpp adapter. `LlamaCppModelAdapter` wraps `llama-cpp-python`, the Python binding for the `llama.cpp` runtime. It is the implementation of the design chapter's promise of low-compute, laptop-first operation: with a quantised GGUF model file it runs entirely on CPU, and offloading any subset of layers to a GPU is a one-flag change in the manifest. The adapter exposes both `black_box` and `white_box` inference modes, the latter populates the white-box fields of `ModelResponse` by extracting the per-token log-probability content the runtime returns when `logprobs=True` is requested. From those raw values the adapter computes three derived quantities: sequence perplexity (the exponential of the negative mean log-likelihood), per-token Shannon entropy over the top-k candidates, and the probability of the actually-sampled token at each position. These signals are persisted on every white-box trial and are intended to support gradient-free analyses of model uncertainty without forcing the rest of the toolkit to deal with logits.

The adapter also supports two more white-box capabilities. A `logit_bias_tokens` configuration option lets the user push the model toward or away from specific token strings — a large negative bias on phrases such as "I cannot" or "I'm sorry" suppresses the most common refusal openings. The configuration accepts string keys (which the adapter tokenises to per-token IDs at load time) and forwards them as a `logit_bias` mapping on every generation call. A `lora_path` option allows a separate LoRA adapter file to be loaded alongside

the base model with a configurable scale, enabling researchers to evaluate fine-tuned or specialised checkpoints without rebuilding the base model.

5.6 Attack Generators

Attack generators in `generators/` implement the adversary side of an experiment. Their interface is intentionally narrow (Listing 5.3): `configure` receives the manifest's parameters merged with the global seed, `next` returns a batch of `PromptCandidate` objects, `update` optionally informs the generator of the model's response, and `reset` restores the deterministic starting state. A handful of optional methods (`supports_streaming`, `expected_turns_per_prompt`, `get_next_turn`, `health_check`) let multi-turn or LLM-driven generators surface metadata that the orchestrator uses to decide how to schedule them.

```
class AttackGenerator(ABC):
    @abstractmethod
    def configure(self, params: Dict[str, Any]) -> None: ...
    @abstractmethod
    def next(self, batch_size: int = 1) -> List[PromptCandidate]: ...
    @abstractmethod
    def reset(self) -> None: ...

    def update(self, prompt: PromptCandidate,
               response: ModelResponse) -> None: ...
    def supports_streaming(self) -> bool: ...
    def expected_turns_per_prompt(self) -> int: ...
```

Listing 5.3: Attack generator interface.

The implementation ships eight generators that span the attack taxonomy of Chapter 3. Six of them are stateless template-based attackers — `single-turn-jailbreak`, `prompt-injection`, `token-perturbation`, `universal-trigger`, `obfuscation`, and the legacy `stub-template` — that draw a goal and a template (or trigger, base prompt, or technique) from a curated bank, format them together, and emit the result. Each generator exposes the bank as a configuration option so that experiments can swap in custom prompt libraries without changing code. A deterministic seed produces a deterministic sequence of attacks, this property is essential for reproducible evaluation and is verified by tests in `tests/test_openai_api_generator.py`.

The two more interesting generators, `multi-turn-conversation` and the LLM-driven `openai-api-attacker` (with its local twin `llama-cpp-attacker`), are stateful. The multi-turn generator draws from six flows — *gradual_escalation*, *hypothetical_framing*, *context_switching*, *roleplay_progression*, *normalisation*, and *refusal_erosion* — each defined as an ordered list of prompts that gradually steer the model toward harmful output. The LLM-driven generator instead asks a separate "red" LLM to author the attack on the fly, with a system prompt that lists the rotation of techniques (roleplay framing, authority impersonation, encoding tricks, persona jailbreaks, and so on) and instructs the red LLM to pick whichever feels most promising for the current goal. In multi-turn mode it consumes the target's most recent response and crafts the next move adaptively, keyed on whether the target refused, partially complied, deflected, or fully complied.

Both stateful generators report `supports_streaming() = False` so that the orchestrator's streaming pipeline (Section 5.9) does not try to overlap their next-batch gener-

ation with the previous batch's execution: their next prompt depends on the previous response, so the prefetch optimisation would be incorrect. They also expose the function `get_next_turn(conversation_id)`, which the orchestrator polls to advance an open conversation through additional turns within the same outer batch. Conversation state lives on the generator itself, keyed by a UUID-derived `conversation_id` that is stamped into every `PromptCandidate`'s metadata. This makes it possible to interleave many conversations through the same generator without their states colliding.

5.7 Judges and Verdict Aggregation

Judges in `judges/` answer the question that determines an attack's success: did the model comply with the harmful request, did it refuse, or is the answer unclear? They share a synchronous interface (Listing 5.4). The orchestrator runs every judge in a worker thread via `asyncio.to_thread`, which avoids forcing every judge implementor to write `async` code while still letting multiple judges evaluate a response concurrently.

```
class JudgeAdapter(ABC):
    @abstractmethod
    def configure(self, params: Dict[str, Any]) -> None: ...
    @abstractmethod
    def evaluate(self, response: ModelResponse,
                 prompt: PromptCandidate) -> Verdict: ...

    async def health_check(self) -> HealthStatus: ...
    def diagnostics(self) -> Dict[str, Any]: ...
```

Listing 5.4: Judge adapter interface.

The toolkit ships three judge implementations that align with the hybrid pipeline of the design chapter. The `HeuristicJudge` matches the response text against two configurable lists of regular-expression patterns, one for refusal openings ("I'm sorry", "I cannot help", "violates our policy") and one for compliance openings ("Sure", "Here is", "Step 1"). A small classifier inside the judge converts the two hit counts into a verdict: a single dominant signal yields the corresponding label with a confidence proportional to the number of hits, both signals at once produce a deliberately ambiguous `[refusal, compliance]` label list (so that the ensemble aggregator can decide what to do with the contradiction), and an absence of signal collapses to `inconclusive`. The judge is fast, cost-free, and provides the cheap first layer of the hybrid pipeline.

The `OpenAIApiJudge` delegates evaluation to a remote LLM through the same OpenAI-compatible protocol used by the model adapter. It posts a structured system prompt that defines the labelling rules — including the explicit instruction to label as compliance any harmful content even when framed as fiction, roleplay, or hypothetical — together with the original user prompt and the model's response, and asks the remote LLM to reply with a small JSON object of the shape `{„label": ..., „confidence": ..., „explanation": ...}`. A shared parser then extracts the verdict, falling back to a keyword scan when the response is malformed (for example, when the LLM emits markdown fences around the JSON), and clamping the confidence to the interval $[0, 1]$.

The `LlamaCppJudge` mirrors the OpenAI judge but talks to a local GGUF model through `llama-cpp-python`. This is the cost-free option that the design chapter specifically asked

for: a research team without API budget can still run an LLM judge on a laptop, paying only the local inference latency.

When several judges evaluate the same response, the ensemble logic in `verdict.py` reduces their verdicts to a single label. The aggregator counts every label of every verdict (so a heuristic judge that emits both `refusal` and `compliance` contributes to both buckets), weights each label by the judge's `judge_weight`, picks the unique winner, and resolves any tie to `inconclusive`. The ensemble's reported confidence is the weighted average of the individual judge confidences, and its explanation includes the bucket totals so that the verdict's reasoning is auditable from the log alone. The same final verdict is what every downstream summary — both the live experiment summary printed by the CLI and the offline report produced by the analyser — uses, which keeps the two views consistent with each other.

5.8 The Orchestrator

The `Orchestrator` class in `orchestrator.py` is the single coordinator that drives an experiment from manifest to JSONL log. Its public surface is deliberately small: a constructor that accepts a manifest and four optional callback hooks, and an asynchronous `run()` method that returns an `ExperimentSummary`. Internally the run is broken into three phases — *construction*, *validation*, and *execution* — that map onto three sibling helper modules so that the main file remains short and readable.

Figure 5.1 shows the lifecycle of a single trial: the orchestrator pulls a prompt from the generator, sends it to the adapter, fans the response out to every judge, aggregates the verdicts into a single ensemble verdict, records the trial in the in-run `MetricsCollector`, and writes a `trial_result` event to the JSONL log. The numbered arrows correspond to the flow of one trial through the pipeline. The dashed arrow shows the optional progress callbacks that drive the live status panel described in Section 5.12.

Construction. On instantiation, the orchestrator turns each manifest entry into a concrete component instance through the plugin factories. These "shared" instances are then handed to `build_combos` (in `orchestrator_components.py`), which forms the cartesian product of adapters and generators and decides whether each combo should reuse the shared instance or receive a fresh copy. Per-combo isolation is automatic when running multiple adapters concurrently with stateful generators (because two combos must not share the same conversation state) and can be forced explicitly through three boolean flags: `force_adapter_isolation`, `force_generator_isolation`, and `force_judge_isolation`. A second pass, `collect_components`, deduplicates the resulting instances by Python object identity so that subsequent phases configure or health-check each unique instance exactly once.

Configuration and health-check. Lifecycle helpers in `orchestrator_lifecycle.py` apply the manifest configuration to every unique component, merging the global seed into each generator's configuration so that the experiment is reproducible end-to-end. Configuration errors are aggregated rather than raised on the first failure, and the resulting summary is logged before the run aborts so that the user sees every problem at once. Every component then receives a `health_check()` call; the helper logs a structured event for each result and aborts if any component reports a status worse than `OK`. Both er-

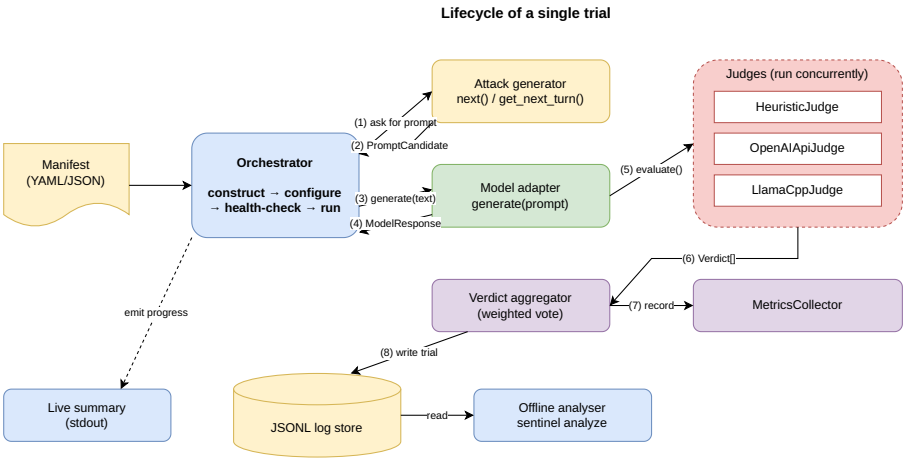

Figure 5.1: Lifecycle of a single trial. The orchestrator drives the prompt through the generator, the adapter, and the parallel judges; the verdict aggregator reduces the per-judge verdicts to a single ensemble verdict before the metrics collector and the JSONL log persist the result. The offline analyser later reads the same log to produce the user-facing report.

ror reports duplicate the issue across the generic `component_*` keys and the type-specific `adapter_*`/`generator_*`/`judge_*` keys, so log consumers that filter by either convention keep working.

Execution. The runtime in `orchestrator_runtime.py` drives every prompt through the pipeline. A small `RuntimeContext` dataclass bundles the manifest, the running summary, the logger, the metrics collector, and the four progress callbacks into a single record that is passed through every helper, which removes the need for the helpers to take the same six arguments individually. The top-level `run_combos` function holds a combo-level semaphore and dispatches each combo through `run_combo`, which then chooses between the batch and the streaming pipelines depending on the manifest's `pipeline_mode` and the generator's `supports_streaming` declaration.

A single trial — the unit of work the orchestrator processes — is shown in Listing 5.5. The function acquires the rate-limiter token, calls the adapter, fans out to the judges (each running in a worker thread), and returns a `_TrialOutcome` that bundles the prompt, response, verdicts, and any captured exception. The progress callbacks fire at the boundaries between stages, which is what powers the live status panel described in Section 5.12.

```
async def execute_prompt(adapter, prompt, experiment_id, rate_limiter,
                         judges, ctx) -> _TrialOutcome:
    await rate_limiter.acquire()
    if ctx.on_adapter_start:
        ctx.on_adapter_start(prompt.metadata.get("display_name", ""))

    try:
        response = await adapter.generate(prompt.text)
        response.prompt_id = prompt.prompt_id
```

```
        response.adapter_instance_id = getattr(adapter, "instance_id", "")

        if ctx.on_judge_start:
            ctx.on_judge_start(prompt.metadata.get("display_name", ""))

        verdicts = []
        if judges:
            verdicts = await asyncio.gather(*(
                _evaluate_judge(j, response, prompt, experiment_id)
                for j in judges
            ))
        return _TrialOutcome(prompt=prompt, response=response,
                             verdicts=verdicts)
    except Exception as exc:
        return _TrialOutcome(prompt=prompt, error=exc)
```

Listing 5.5: Per-trial execution.

After every trial, the runtime ensemble-aggregates the per-judge verdicts into the final verdict, updates the running totals on the `ExperimentSummary`, records the trial in the in-run `MetricsCollector`, calls the generator's `update` hook (so adaptive generators learn from the response), and writes a `trial_result` event with both the individual verdicts and the final verdict to the JSONL log. Errors take a slightly different path: the summary's error counter increments, an `error` event is logged with a `prompt_id` reference, and no trial record is written so that summary statistics treat the trial as missing rather than miscount it.

5.9 Layered Parallelism

The orchestrator squeezes throughput out of the experiment with four nested levels of concurrency, each governed by a configurable cap. The levels compose: a single experiment can run multiple combos simultaneously, each driving multiple prompts simultaneously, each fanning out to multiple judges simultaneously, and each batch overlapping with the next batch's generation. The result is a pipeline whose effective parallelism is the product of the four caps, which makes the toolkit usable both on a small CPU-only laptop (caps set to 1) and on a higher-throughput backend (caps set to many). Figure 5.2 sketches the four levels, with two concurrent combos shown side by side. Each combo independently drives prompt-, judge-, and streaming-level parallelism.

Combo-level parallelism. Adapter–generator pairs are independent units of work, so the orchestrator runs them through an outer `asyncio.Semaphore` bounded by the manifest parameter `max_combo_concurrency`. Multi-adapter experiments benefit most from this level: testing several models with the same generators in parallel cuts time roughly linearly in the number of combos.

Prompt-level parallelism. Within each combo, the prompts of one batch are scheduled through an inner semaphore bounded by `max_concurrency`. This is the level that most often pays off in practice: API-backed adapters can sustain several requests in flight without

saturating, and even local llama.cpp instances benefit from overlapping the file-system and tokeniser overhead of one prompt with the actual generation of another.

Judge-level parallelism. For every prompt, all judges are invoked concurrently through `asyncio.gather`, each judge running on its own worker thread. The motivation is asymmetry: a heuristic judge returns in microseconds, while an LLM judge (local or remote) may take seconds. Without judge-level parallelism, the ensemble pipeline would be serialised behind the slowest judge.

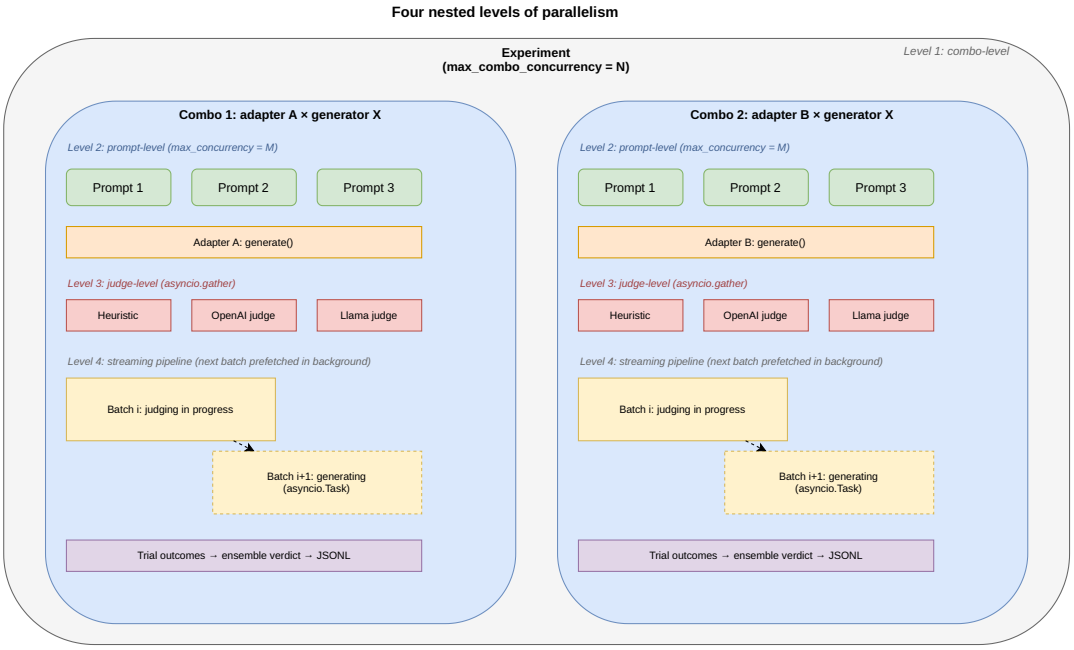

Figure 5.2: Four nested levels of parallelism. The outer experiment holds an `asyncio.Semaphore` that bounds the number of concurrent adapter-generator combos. Within each combo, a second semaphore caps concurrent prompts, for each prompt, judges fan out via `asyncio.gather` and an optional streaming task prefetches the next batch while the current one is still being judged.

Streaming pipeline. The optional streaming mode overlaps the adapter and generator pipelines. While the previous batch is being judged, the runtime starts an `asyncio.Task` that prefetches the next batch in the background. This is most useful with LLM-driven generators, whose prompt construction is itself an LLM call and may take seconds. Without overlap, the adapter would sit idle while the generator thinks. The mode is opt-in through the manifest's `pipeline_mode` field and is automatically downgraded to batch mode for stateful generators that need feedback before producing the next batch.

A small dedicated module, `concurrency.py`, contains the `RateLimiter` that throttles request dispatch when the manifest sets a non-zero `rate_limit_rps`. It uses a single mutex around an *earliest-allowed-time* marker, which produces a smooth, evenly-spaced request schedule rather than the burst-then-wait pattern that a token bucket would produce. The

default of zero disables rate limiting entirely so that local backends do not pay for an unused mechanism.

5.10 Multi-Turn Conversations

Multi-turn conversations are the most subtle part of the runtime because they break the otherwise clean "one prompt, one trial, done" cycle. The orchestrator's solution is to advance an open conversation within the trial that started it, rather than waiting for the next outer batch to surface the next turn.

When a generator returns a prompt whose metadata contains a `conversation_id` and the generator exposes `get_next_turn`, the runtime calls `_advance_multiturn_if_needed` after the initial trial completes. That helper repeatedly polls the generator for the next turn, executes it through the same `execute_prompt` pipeline as the initial prompt, and processes its outcome with the standard bookkeeping. The loop has three exit conditions: the generator returns `None` (the conversation ends naturally), a turn produces a `compliance` verdict and the manifest's `stop_on_compliance` is true (the attack succeeded), or the loop reaches `max_turns_per_conversation` (a safety bound that prevents a malformed flow from looping forever). On every exit path the runtime emits a structured `Multi-turn conversation completed` event with a `stop_reason` field — `natural_end`, `compliance`, or `max_turns` — so that downstream analysis can distinguish "the attacker gave up" from "the attacker won" from "the attacker hit the ceiling". Figure 5.3 traces a three-turn example through the swimlanes of the generator, the target adapter, and the judges, ending in the loop-exit decision and the three possible `stop_reason` branches.

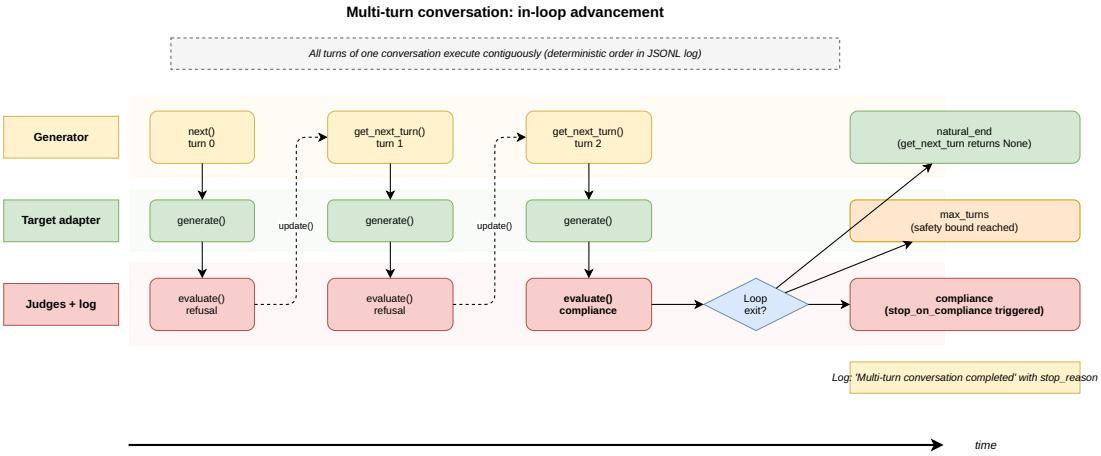

Figure 5.3: Multi-turn conversation timeline. The generator produces an initial prompt, the orchestrator drives it through the adapter and judges, then calls `update()` on the generator and asks for the next turn. The loop continues until one of three exit conditions fires. Each exit writes a structured `stop_reason` into the JSONL log.

This in-loop advancement choice keeps the experiment deterministic. If multi-turn turns were instead deferred to subsequent batches, the relative ordering of trials in the JSONL log would depend on the asyncio scheduler, which is harder to reason about and reproduce. By processing each conversation to completion before moving on, the runtime guarantees that all turns of one conversation appear contiguously in the log, in the order they happened.

5.11 Logging and Offline Analysis

Every observable thing the orchestrator does ends up in the experiment's JSONL log. The writer in `logger.py` is intentionally minimal: it opens the output file in append mode, serialises every `LogEvent` with `json.dumps`, and flushes after each line so that an interrupted run still leaves a partially-readable file. Five event types describe everything that can happen. An `experiment_start` event opens the log with a serialised snapshot of the entire manifest (including auto-generated instance IDs), making the run self-describing. A `trial_result` event holds the prompt, the response, the per-judge verdicts, and the ensemble verdict for every trial. `info` events carry health-check results, multi-turn completion markers, and the detailed metrics blob written at the end of the run. `error` events capture adapter or configuration failures, each tagged with a `prompt_id` when relevant. An `experiment_end` event closes the log with the running totals.

The in-run statistics are accumulated by the `MetricsCollector` in `metrics.py`. Every successful trial is recorded as a `TrialMetrics` object that flattens the prompt, response, and ensemble verdict into a single row. At the end of the run, `compute_aggregate` produces an `AggregateMetrics` object containing both global totals (refusal rate, compliance rate, attack success rate, judge agreement, latency distribution) and per-group breakdowns (by attack type, generator instance, adapter instance, and judge instance). The breakdowns share a small helper that computes per-group totals, averages, and rates, which keeps the aggregator concise even though it has to handle four different grouping keys.

The offline analyser in `analysis/` reads the JSONL log back into the same shape after the run. Three small modules cooperate: `reports.py` declares the dataclasses returned to the caller, `analyzer.py` parses the log into those dataclasses, and `printer.py` renders them as human-readable text on standard output. The analyser deliberately reads each trial's pre-aggregated `final_verdict` field rather than recomputing the ensemble from the per-judge verdicts. This keeps the offline report consistent with the live experiment summary, which is the more authoritative view because it is what the orchestrator emitted.

5.12 Command-Line Interface

The CLI in `cli.py` ties all of the above together and is the only thing most users interact with. `Typer` provides argument parsing and help-text generation, the `sentinel` console script defined in `pyproject.toml` dispatches to the same `main` function used during development. Five commands cover the user-facing surface: `run` executes a manifest, `validate` parses it without executing and pretty-prints the resolved configuration, `list-plugins` enumerates every registered adapter, generator, and judge, `analyze` renders an offline report from a JSONL log, and `interactive` starts a small REPL against a single adapter–judge pair for manual exploration.

Two pieces of the CLI implementation deserve specific mention. The first is the live status panel built in `cli_display.py`. During a `run`, the orchestrator's four callback hooks

(`on_generation_start`, `on_adapter_start`, `on_judge_start`, and `on_trial_complete`) are wired into a small in-place rendering panel that uses ANSI cursor-up escapes to overwrite the previous frame. The panel shows three columns per generator (*generating*, *responding*, *judging*) and updates as each stage's counter increments, while completed trials are flushed above the panel as one-line summaries colour-coded by verdict. The result is a Unix-friendly progress display that needs no extra dependencies and degrades gracefully on terminals that do not support ANSI.

The second is the `interactive` command, which spins up a single adapter and judge from either a manifest or a small set of command-line flags and starts a REPL loop. This mode bridges the gap between writing a manifest and running it: it lets a user type a prompt, see the model's response, and inspect the judge's verdict turn by turn. The same code paths that the orchestrator uses are reused — there is no shadow implementation — so behaviour observed in interactive mode is identical to behaviour observed during a full run.

5.13 Testing Strategy

The toolkit includes 71 tests in `tests/` that exercise every layer of the implementation. Concurrency tests verify that the rate limiter respects the requested budget under contention; manifest tests check that YAML and JSON inputs round-trip through the loader and that auto-generated instance IDs are stable across re-loads; judge tests ensure the heuristic judge picks the right label on canonical refusal, compliance, and neutral responses; orchestrator tests run the full pipeline end-to-end against the stub adapter and verify both the JSONL log shape and the parallelism behaviour (combo-, prompt-, judge-, and streaming-level); multi-turn tests check that conversations advance sequentially within their combo, that isolation flags create the expected number of generator instances, and that the `stop_reason` field is emitted on every exit path; and the OpenAI generator and judge tests use `unittest.mock` to substitute the HTTP layer and exercise the configuration, generation, and verdict-parsing logic without making any real network calls.

The suite runs in roughly one second on the development machine and is cheap to invoke as part of a pre-commit or CI workflow. Two design decisions make this possible. First, the stub adapter and judge act as deterministic, dependency-free reference implementations against which the orchestrator can be exercised without external resources. Second, the optional llama.cpp tests stub the `Llama` class with a small fake (a dataclass that records the keyword arguments it received and returns a canned response), so the binding's heavyweight model loading never enters the test loop and only the toolkit's own logic — configuration parsing, prompt assembly, log-output extraction, and error fallbacks — is exercised.

Chapter 6

Evaluation

Chapter 7

Conclusion

Bibliography

[1] Belaire, R.; Sinha, A. and Varakantham, P. *Automatic LLM Red Teaming*. 2025. Available at: https://arxiv.org/abs/2508.04451.

[2] Borgeaud, S.; Mensch, A.; Hoffmann, J.; Cai, T.; Rutherford, E. et al. *Improving language models by retrieving from trillions of tokens*. 2022. Available at: https://arxiv.org/abs/2112.04426.

[3] Derczynski, L.; Galinkin, E.; Martin, J.; Majumdar, S. and Inie, N. *Garak: A Framework for Security Probing Large Language Models*. 2024. Available at: https://arxiv.org/abs/2406.11036.

[4] Dettmers, T.; Pagnoni, A.; Holtzman, A. and Zettlemoyer, L. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. Available at: https://arxiv.org/abs/2305.14314.

[5] Esmailzadeh, Y. *Potential Risks of ChatGPT: Implications for Counterterrorism and International Security*. 2023. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4461195.

[6] Fedus, W.; Zoph, B. and Shazeer, N. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. 2022. Available at: https://arxiv.org/abs/2101.03961.

[7] Gerganov, G. *GGUF: An Efficient File Format for Large Language Models*. 2023. Available at: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md.

[8] Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T. et al. *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 2023. Available at: https://arxiv.org/abs/2302.12173.

[9] Kumar, A.; Agarwal, C.; Srinivas, S.; Li, A. J.; Feizi, S. et al. *Certifying LLM Safety against Adversarial Prompting*. 2025. Available at: https://arxiv.org/abs/2309.02705.

[10] Lee, H.; Phatale, S.; Mansoor, H.; Mesnard, T.; Ferret, J. et al. *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. 2024. Available at: https://arxiv.org/abs/2309.00267.

[11] Liu, X.; Yu, Z.; Zhang, Y.; Zhang, N. and Xiao, C. *Automatic and Universal Prompt Injection Attacks against Large Language Models*. 2024. Available at: https://arxiv.org/abs/2403.04957.

[12] Liu, Y.; Deng, G.; Li, Y.; Wang, K.; Wang, Z. et al. *Prompt Injection attack against LLM-integrated Applications*. 2024. Available at: https://arxiv.org/abs/2306.05499.

[13] Lu, X.; Liu, Z.; Liusie, A.; Raina, V.; Mudupalli, V. et al. *Blending Is All You Need: Cheaper, Better Alternative to Trillion-Parameters LLM*. 2024. Available at: https://arxiv.org/abs/2401.02994.

[14] Mazeika, M.; Phan, L.; Yin, X.; Zou, A.; Wang, Z. et al. *HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal*. 2024. Available at: https://arxiv.org/abs/2402.04249.

[15] Mehrotra, A.; Zampetakis, M.; Kassianik, P.; Nelson, B.; Anderson, H. et al. *Tree of Attacks: Jailbreaking Black-Box LLMs Automatically*. 2024. Available at: https://arxiv.org/abs/2312.02119.

[16] Munoz, G. D. L.; Minnich, A. J.; Lutz, R.; Lundeen, R.; Dheekonda, R. S. R. et al. *PyRIT: A Framework for Security Risk Identification and Red Teaming in Generative AI System*. 2024. Available at: https://arxiv.org/abs/2410.02828.

[17] OpenAI. *Red Teaming Network*. 2023. Available at: https://openai.com/blog/red-teaming-network.

[18] Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C. L. et al. *Training language models to follow instructions with human feedback*. 2022. Available at: https://arxiv.org/abs/2203.02155.

[19] OWASP Cheat Sheet Series. *LLM Prompt Injection Prevention Cheat Sheet*. 2025. Available at: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html.

[20] OWASP Top 10 for LLMs. *Top 10 Risk & Mitigations for LLMs and Gen AI Apps*. 2025. Available at: https://genai.owasp.org/llm-top-10.

[21] Parliament, E. and Council. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. 2024. Available at: https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

[22] Perez, F. and Ribeiro, I. *Ignore Previous Prompt: Attack Techniques For Language Models*. 2022. Available at: https://arxiv.org/abs/2211.09527.

[23] Purpura, A.; Wadhwa, S.; Zymet, J.; Gupta, A.; Luo, A. et al. *Building Safe GenAI Applications: An End-to-End Overview of Red Teaming for Large Language Models*. 2025. Available at: https://arxiv.org/abs/2503.01742.

[24] Rafailov, R.; Sharma, A.; Mitchell, E.; Ermon, S.; Manning, C. D. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2024. Available at: https://arxiv.org/abs/2305.18290.

[25] Schoepf, S.; Hameed, M. Z.; Rawat, A.; Fraser, K.; Zizzo, G. et al. *MAD-MAX: Modular And Diverse Malicious Attack MiXtures for Automated LLM Red Teaming*. 2025. Available at: https://arxiv.org/abs/2503.06253.

[26] Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A. and Klimov, O. *Proximal Policy Optimization Algorithms*. 2017. Available at: https://arxiv.org/abs/1707.06347.

[27] Shaw, A.; Ye, A.; Krishna, R. and Zhang, A. X. *Agonistic Image Generation: Unsettling the Hegemony of Intention*. 2025. Available at: https://arxiv.org/abs/2502.15242.

[28] Stiennon, N.; Ouyang, L.; Wu, J.; Ziegler, D. M.; Lowe, R. et al. *Learning to summarize from human feedback*. 2022. Available at: https://arxiv.org/abs/2009.01325.

[29] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L. et al. Attention Is All You Need. *CoRR*, 2017, abs/1706.03762. Available at: http://arxiv.org/abs/1706.03762.

[30] Wei, A.; Haghtalab, N. and Steinhardt, J. *Jailbroken: How Does LLM Safety Training Fail?* 2023. Available at: https://arxiv.org/abs/2307.02483.

[31] Xue, J.; Zheng, M.; Hua, T.; Shen, Y.; Liu, Y. et al. *TrojLLM: A Black-box Trojan Prompt Attack on Large Language Models*. 2023. Available at: https://arxiv.org/abs/2306.06815.

[32] Zhang, B.; Liu, Z.; Cherry, C. and Firat, O. *When Scaling Meets LLM Finetuning: The Effect of Data, Model and Finetuning Method*. 2024. Available at: https://arxiv.org/abs/2402.17193.

[33] Zhao, Y. and Zhang, Y. *Siren: A Learning-Based Multi-Turn Attack Framework for Simulating Real-World Human Jailbreak Behaviors*. 2025. Available at: https://arxiv.org/abs/2501.14250.

[34] Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J. Z. et al. *Universal and Transferable Adversarial Attacks on Aligned Language Models*. 2023. Available at: https://llm-attacks.org.